

CSE 4403: Algorithms

Lecture02

Sorting Algorithms -1

Anika Farzana

Junior Lecturer

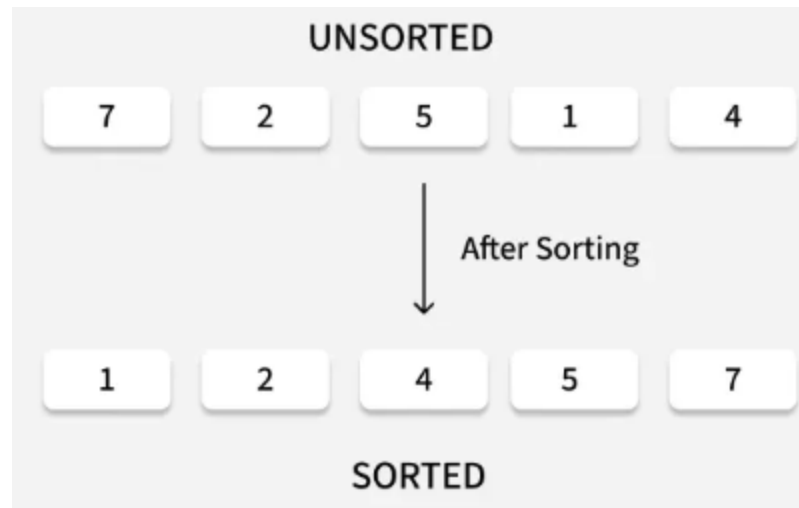
Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Sorting

- Rearranging items in an array or list in a specified order: ascending or descending
- Key Terms:
 - In-place Sorting
 - Stable Sorting



Sorting Algorithms

1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Counting Sort
5. Radix Sort
6. Merge Sort
7. Quick Sort

and so on..

Basics

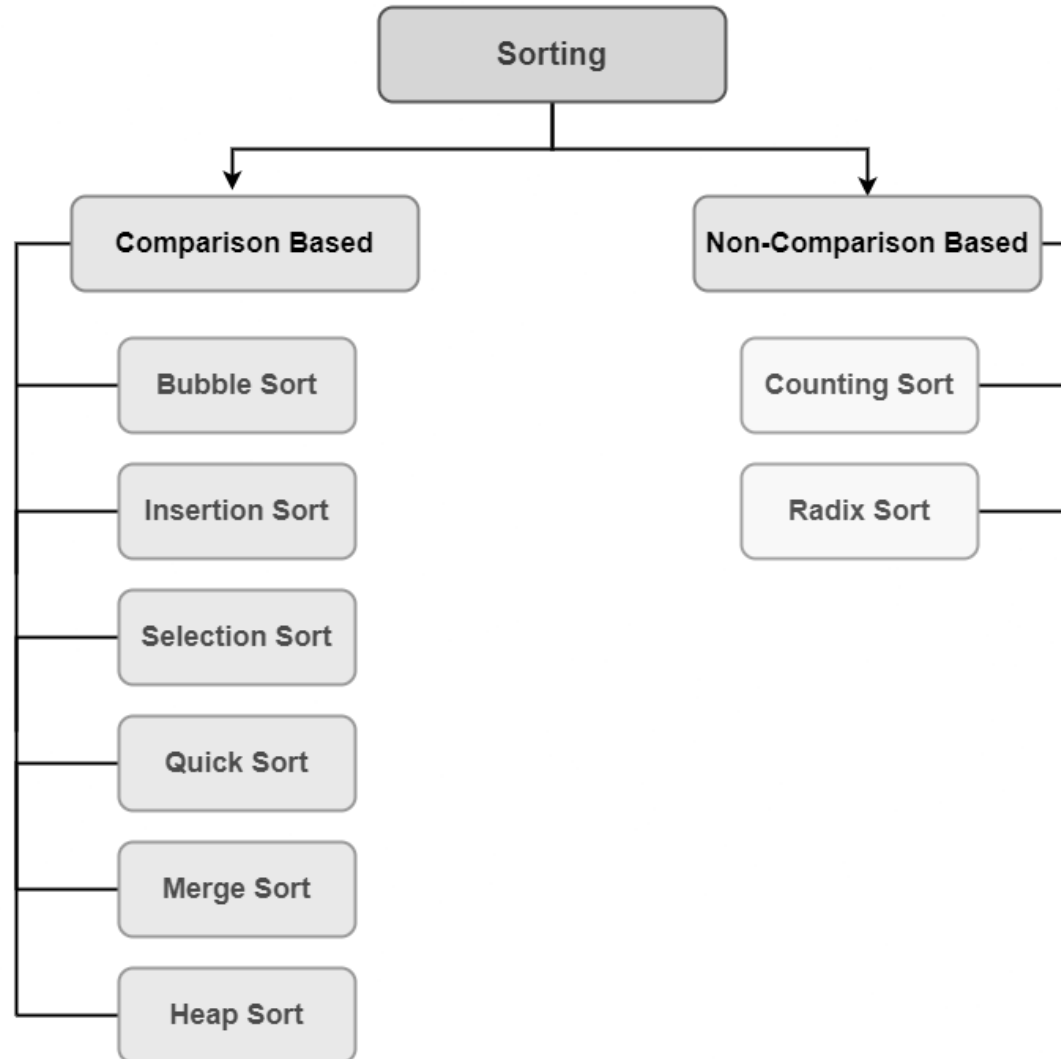
In-place sort:

- A sorting algorithm that uses **constant space** for producing the output (modifies the given array only).
- Example: Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Heap Sort.
- MergeSort, Counting Sort, Radix Sort are not in-place.

Stable sort:

- When two same items appear in the **same order** in sorted data as in the original.
- Example: Bubble Sort, Insertion Sort, Counting Sort, Merge Sort.
- Selection Sort, QuickSort, HeapSort are not stable.

Types



Bubble Sort

- Repeatedly swaps adjacent elements if they are in the wrong order
- Multiple passes
- **In each pass, the largest unsorted element comes to the correct position**
- The data elements "**bubble**" to their correct positions in a list, much like air bubbles rising to the surface of water.

(Larger bubbles generally rise faster than smaller bubbles in a liquid like water because they have a higher buoyant force)



Bubble Sort : Code

```
void bubbleSort(vector<int>& arr) {  
    int n = arr.size();  
  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                swap(arr[j], arr[j + 1]);  
            }  
        }  
    }  
  
    .....  
}
```

Bubble Sort : An example

Array: 5, 6, 1, 3

**Sort the array in ascending order
Use Bubble Sort**

How many passes we need?

Bubble Sort : An example

For **i = 0 (first pass)**, initial array: 5, 6, 1, 3

j runs from 0 until $(n - i - 1) = (4 - 0 - 1) = 3$

So we need 3 swaps at max

Index	Array	Action to take
j = 0	5, 6, 1, 3	No action
j = 1	5, 6, 1, 3	Swap 6, 1
j = 2	5, 1, 6, 3	Swap 6, 3
	5, 1, 3, 6	

Array for next iteration: 5, 1, 3, 6



6 bubbled to the
correct position at
the end of first pass

Bubble Sort : An example

For $i = 1$ (**second pass**), initial array: 5, 1, 3, 6

j runs from 0 until $(n - i - 1) = (4 - 1 - 1) = 2$

So we need 2 swaps at max

Index	Array	Action to take
$j = 0$	5, 1, 3, 6	Swap 5, 1
$j = 1$	1, 5, 3, 6	Swap 5, 3
	1, 3, 5, 6	

Array for next iteration: 1, 3, 5, 6



Now both 5 and 6 are in the correct position after second pass

Bubble Sort : Code

```
void bubbleSort(vector<int>& arr) {  
    int n = arr.size();  
  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                swap(arr[j], arr[j + 1]);  
            }  
        }  
    }  
  
    .....  
}
```

Bubble Sort : Complexity

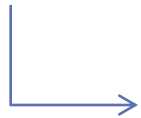
Time Complexity: $O(n^2)$ for best, average and worst cases

Auxiliary Space: $O(1)$

Bubble Sort : Complexity

Time Complexity: $O(n^2)$ for best, average and worst cases

Auxiliary Space: $O(1)$

 In place sorting

Bubble Sort : Optimized Code

```
void bubbleSort(vector<int>& arr) {  
    int n = arr.size();  
    bool swapped;  
  
    for (int i = 0; i < n - 1; i++) {  
        swapped = false;  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                swap(arr[j], arr[j + 1]);  
                swapped = true;  
            }  
        }  
  
        // If no two elements were swapped, then break  
        if (!swapped)  
            break;  
    }  
}
```

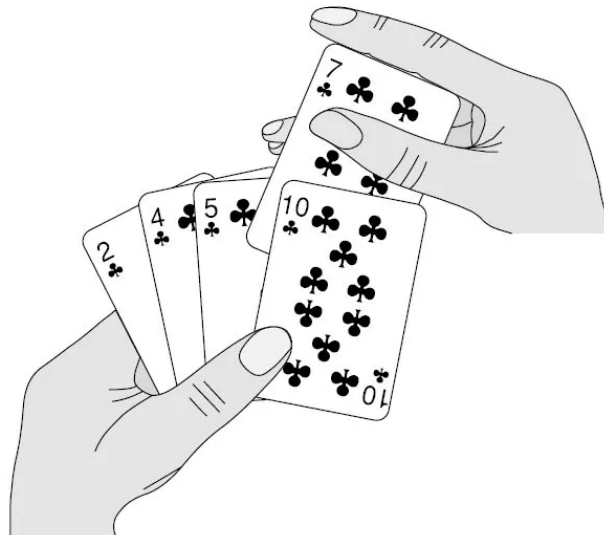
Bubble Sort : Complexity for Improved Code

Time Complexity: $O(n)$ for best case when array is already sorted, $O(n^2)$ for average and worst cases

Auxiliary Space: $O(1)$

Insertion Sort

- Builds the sorted array one element at a time.
- Its like a pack of playing cards in your hand, where you insert a new card into its correct position starting from the beginning



Insertion Sort : Code

```
void insertionsort(vector<int>& arr, int n) {  
    for (int i = 1; i < n; ++i) {  
        int key = arr[i];  
        int j = i - 1;  
  
        // move elements greater than key one position ahead  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
        arr[j + 1] = key;  
    }  
}
```

Insertion Sort: An example

Array: 12, 11, 13, 5, 6

**Sort the array in ascending order
Use Insertion Sort**

How many passes we need?

Insertion Sort: An example

For $i = 1$ (index of first element to move front),
initial array: 12, 11, 13, 5, 6

Index	Array	Action to take
	12, 11 , 13, 5, 6	Key = 11
$j = 0$	<u>12</u> , 12, 13, 5, 6 	Copy j th element to $(j+1)$ th position since $\text{arr}[j] > \text{arr}[\text{key}]$; Decrement j
$j = -1$	12, 12, 13, 5, 6	$j < 0$ so break
$j = -1$	11, 12, 13, 5, 6	Place the key at $j+1$

Array for next iteration: 11, 12, 13, 5, 6



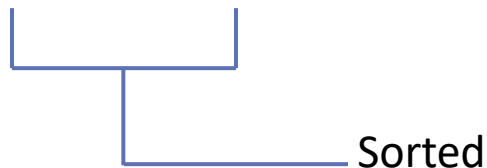
Insertion Sort: An example

For $i = 2$,

initial array: 11, 12, 13, 5, 6

Index	Array	Action to take
	11, 12, 13 , 5, 6	Key = 13
$j = 1$	11, <u>12</u> , 13, 5, 6	Since $arr[j] < arr[j+1]$ break
$j = 1$	11, 12, 13, 5, 6	Place the key at $j+1$

Array for next iteration: 11, 12, 13, 5, 6



Insertion Sort: An example

For $i = 3$,

initial array: 11, 12, 13, 5, 6

Index	Array	Action to take
	11, 12, 13, 5 , 6	Key = 5
$j = 2$	11, 12, <u>13</u> , 13, 6	Copy j th element to $(j+1)$ th position since $arr[j] > arr[key]$; Decrement j
$j = 1$	11, <u>12</u> , 12, 13, 6	„
$j = 0$	<u>11</u> , 11, 12, 13, 6	„
$j = -1$	11, 11, 12, 13, 6	$j < 0$, so break
$j = -1$	5, 11, 12, 13, 6	Place the key at $j+1$

Array for next iteration: 5, 11, 12, 13, 6



Insertion Sort: An example

For $i = 4$, Similarly

initial array: 5, 11, 12, 13, 6

Index	Array	Action to take
	5, 11, 12, 13, 6	Key = 6
$j = 3$	5, 11, 12, <u>13</u> , 13	Copy j th element to $(j+1)$ th position since $arr[j] > arr[key]$; Decrement j
$j = 2$	5, 11, <u>12</u> , 12, 13	„
$j = 1$	5, <u>11</u> , 11, 12, 13	„
$j = 0$	<u>5</u> , 11, 11, 12, 13	Since $arr[j] < arr[j+1]$ break
$j = 0$	5, 6, 11, 12, 13	Place the key at $j+1$

Sorted array: 5, 6, 11, 12, 13

Insertion Sort : Complexity

Time Complexity: $O(n)$ for best case when array is already sorted, $O(n^2)$ for average and worst cases

Auxiliary Space: $O(1)$

Insertion sort is preferred when the array size is small or nearly sorted. It is better than bubble sort since no swapping operation is done

Selection Sort

- Repeatedly selects the smallest (or largest) element from the unsorted part of the array and swaps it with the first unsorted element.
- After the i -th pass, we get a sorted array from index 0 to index i .
- This process continues until the array is fully sorted.
- Let's see the process for sorting the array in ascending order.

Selection Sort : Code

```
void selectionSort(vector<int> &arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; ++i) {

        // Assume the current position holds
        // the minimum element
        int min_idx = i;

        // Iterate through the unsorted portion
        // to find the actual minimum
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[min_idx]) {

                // Update min_idx if a smaller
                // element is found
                min_idx = j;
            }
        }

        // Move minimum element to its
        // correct position
        swap(arr[i], arr[min_idx]);
    }
}
```

Selection Sort : An example

For $i = 0$ (Let this index contains the min element),
initial array: 12, 11, 13, 5

Index	Array	Action to take
	<u>12</u> , 11, 13, 5	min_id = 0
j = 1	12, <u>11</u> , 13, 5	min_id = 1
j = 2	12, 11, <u>13</u> , 5	min_id = 1
j = 3	12, 11, 13, <u>5</u>	min_id = 3
	5, 11, 13, 12	Swap arr[i] and arr[min_idx]

Array for next iteration: 5, 12, 13, 5, 6

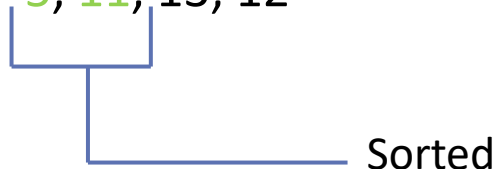
↑
Sorted

Selection Sort : An example

For $i = 1$ (Let this index contains the min element),
initial array: 5, 11, 13, 12

Index	Array	Action to take
	5, 11, 13, 12	min_id = 1
$j = 2$	5, 11, <u>13</u> , 12	min_id = 1
$j = 3$	5, 11, 13, <u>12</u>	min_id = 1
	5, 11, 13, 12	Swap arr[i] and arr[min_idx]

Array for next iteration: 5, 11, 13, 12



Selection Sort : An example

For $i = 2$ (Let this index contains the min element),
initial array: 5, 11, 13, 12

Index	Array	Action to take
	5, 11, 13, 12	min_id = 2
$j = 2$	5, 11, 13, 12	min_id = 2
$j = 3$	5, 11, 13, 12	min_id = 3
	5, 11, 12, 13	Swap arr[i] and arr[min_idx]

Sorted Array: 5, 11, 12, 13

Selection Sort : Complexity

Time Complexity: $O(n^2)$ for all cases

Auxiliary Space: $O(1)$

Counting Sort

- A non-comparison-based sorting algorithm.
- Works efficiently when the range of input values is small relative to the number of elements.
- Counts the frequency of each distinct element.
- Uses that count to place elements directly into their correct sorted positions.
- A stable sorting algorithm (We will see why)

Counting Sort : Code

```
vector<int> countsort(vector<int>& arr) {  
    int n = arr.size();  
  
    // find the maximum element  
    int maxval = 0;  
    for (int i = 0; i < n; i++)  
        maxval = max(maxval, arr[i]);  
  
    // create and initialize count array  
    vector<int> count(maxval + 1, 0);  
  
    // count frequency of each element  
    for (int i = 0; i < n; i++)  
        count[arr[i]]++;  
  
    // compute prefix sum  
    for (int i = 1; i <= maxval; i++)  
        count[i] += count[i - 1];  
  
    // build output array  
    vector<int> ans(n);  
    for (int i = n - 1; i >= 0; i--) {  
        ans[count[arr[i]] - 1] = arr[i];  
        count[arr[i]]--;  
    }  
  
    return ans;  
}
```

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---

Max Val = 4

So, size of count array
= $4 + 1 = 5$

0 1 2 3 4

Count Array:

1	2	2	0	1
---	---	---	---	---

**Cumulative count array/
Prefix Sum count array:**

0 1 2 3 ④

1	3	5	5	6
---	---	---	---	---



No. of elements before and including 4
in sorted order {0,1,1,2,2,4}

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---

Max Val = 4

So, size of count array
= $4 + 1 = 5$

Count Array:

0	1	2	3	4
1	2	2	0	1

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4
1	3	5	5	6



No. of elements before and including 3 in
sorted order {0,1,1,2,2}

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---

Max Val = 4

So, size of count array
= $4 + 1 = 5$

0 1 2 3 4

Count Array:

1	2	2	0	1
---	---	---	---	---

**Cumulative count array/
Prefix Sum count array:**

0 1 2 3 4

1	3	5	5	6
---	---	---	---	---



No. of elements before and including 2 in
sorted order {0,1,1,2,2}

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

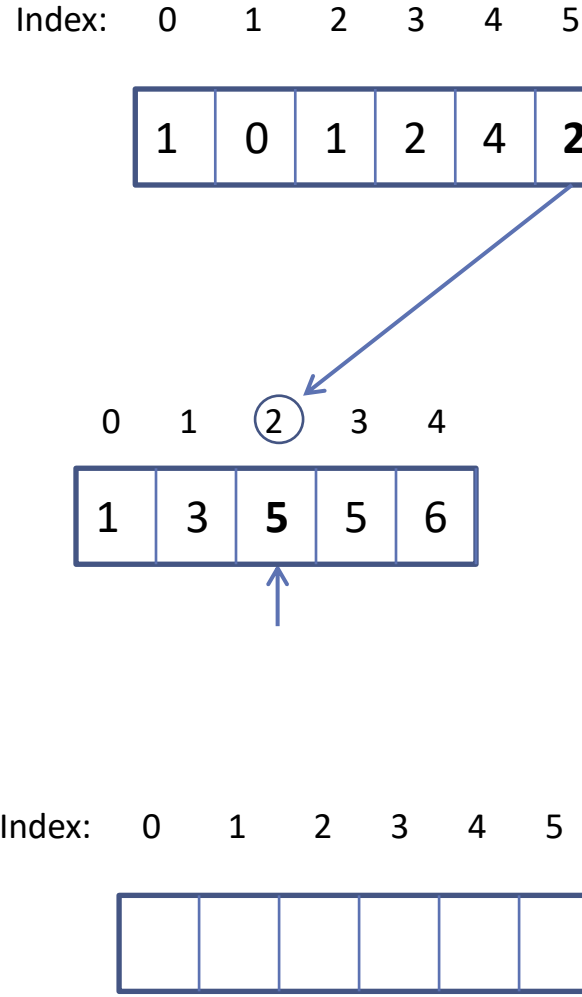
1	0	1	2	4	2
---	---	---	---	---	---

Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0 1 2 3 4

1	3	5	5	6
---	---	---	---	---



Index: 0 1 2 3 4 5

Result array:

--	--	--	--	--	--

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---

Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4
1	3	5	5	6

Place 2 in index $5-1=4$

Index: 0 1 2 3 4 5

Result array:

				2	
--	--	--	--	---	--

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---



Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4
1	3	4	5	6



Decrement prefixCnt[2]

Index: 0 1 2 3 4 5

Result array:

				2	
--	--	--	--	---	--

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	----------	---

Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4	
1	3	4	5	6	

Index: 0 1 2 3 4 5

Result array:

				2	
--	--	--	--	---	--

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	----------

Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4
1	3	4	5	6

Place 4 in index $6-1=5$

Index: 0 1 2 3 4 5

Result array:

				2	4
--	--	--	--	---	----------

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---

Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4
1	3	4	5	5

Decrement prefixCnt of 4

Index: 0 1 2 3 4 5

Result array:

				2	4
--	--	--	--	---	---

Counting Sort : Complexity

Time Complexity: $O(n + m)$, where n and m are the size of `arr[]` and `count[]` respectively

Auxiliary Space: $O(n + m)$

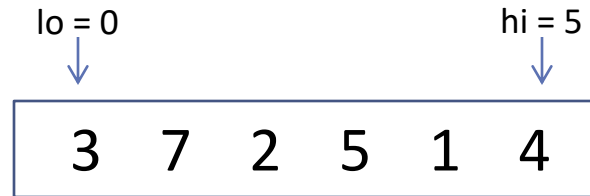
Merge Sort

- Popular sorting algorithm
- Known for its efficiency and stability.
- It follows the Divide and Conquer approach.
- Works by-
 - recursively dividing the input array into two halves,
 - recursively sorting the two halves and
 - finally merging them back together to obtain the sorted array.

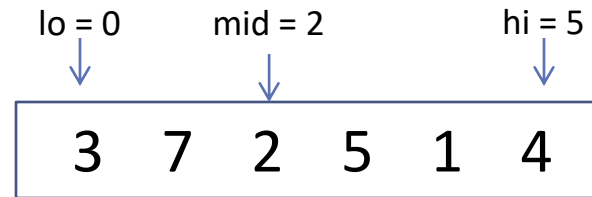
Merge Sort : Code

```
void mergeSort(int arr[], int lo, int hi) {  
    if (lo < hi) {  
        int mid = lo + (hi - lo) / 2;  
  
        mergeSort(arr, lo, mid);  
        mergeSort(arr, mid + 1, hi);  
  
        merge(arr, lo, mid, hi);  
    }  
}
```

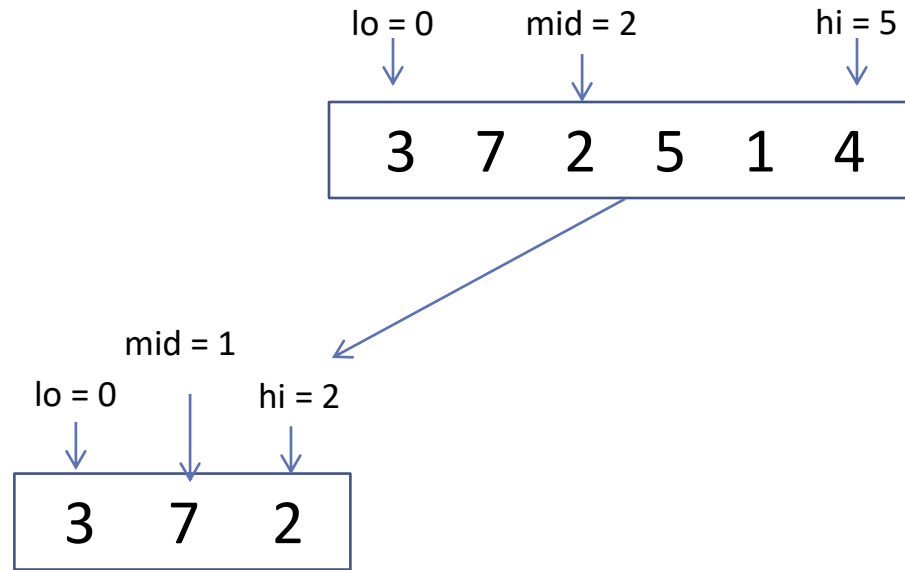
Merge Sort : An example



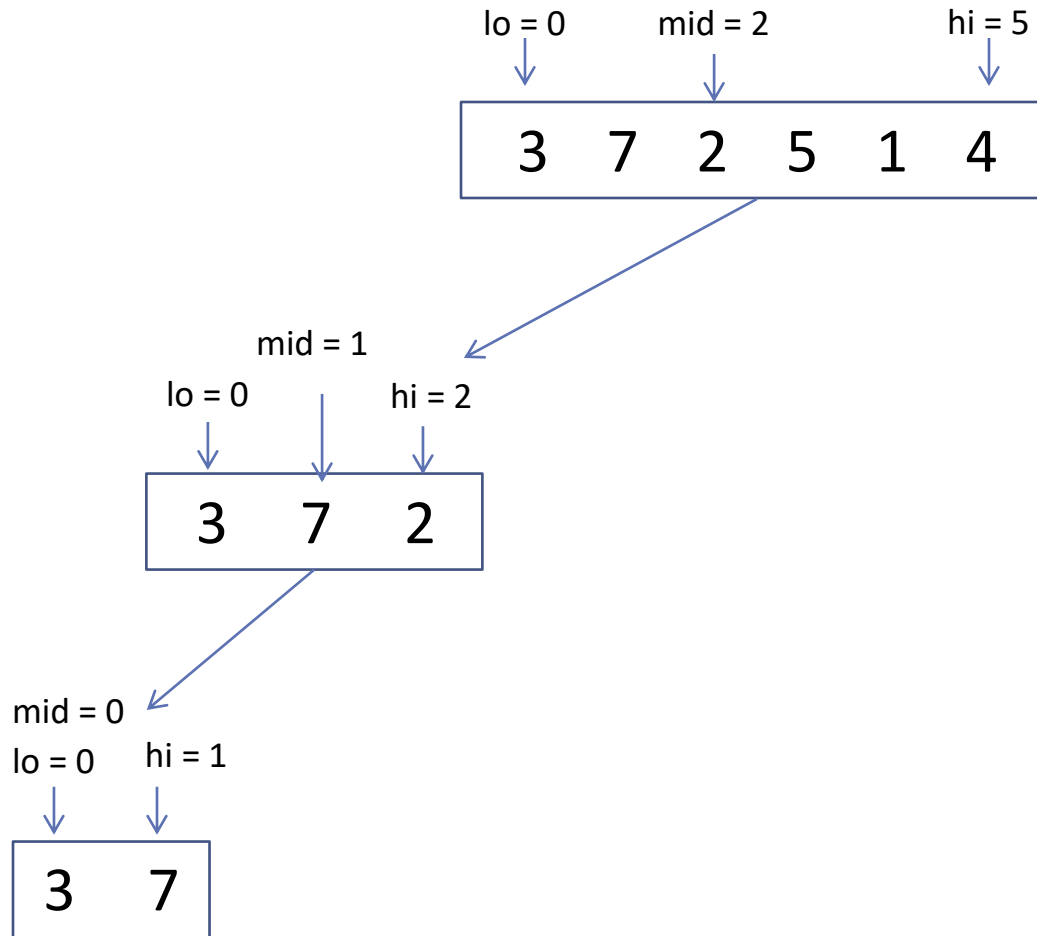
Merge Sort : An example



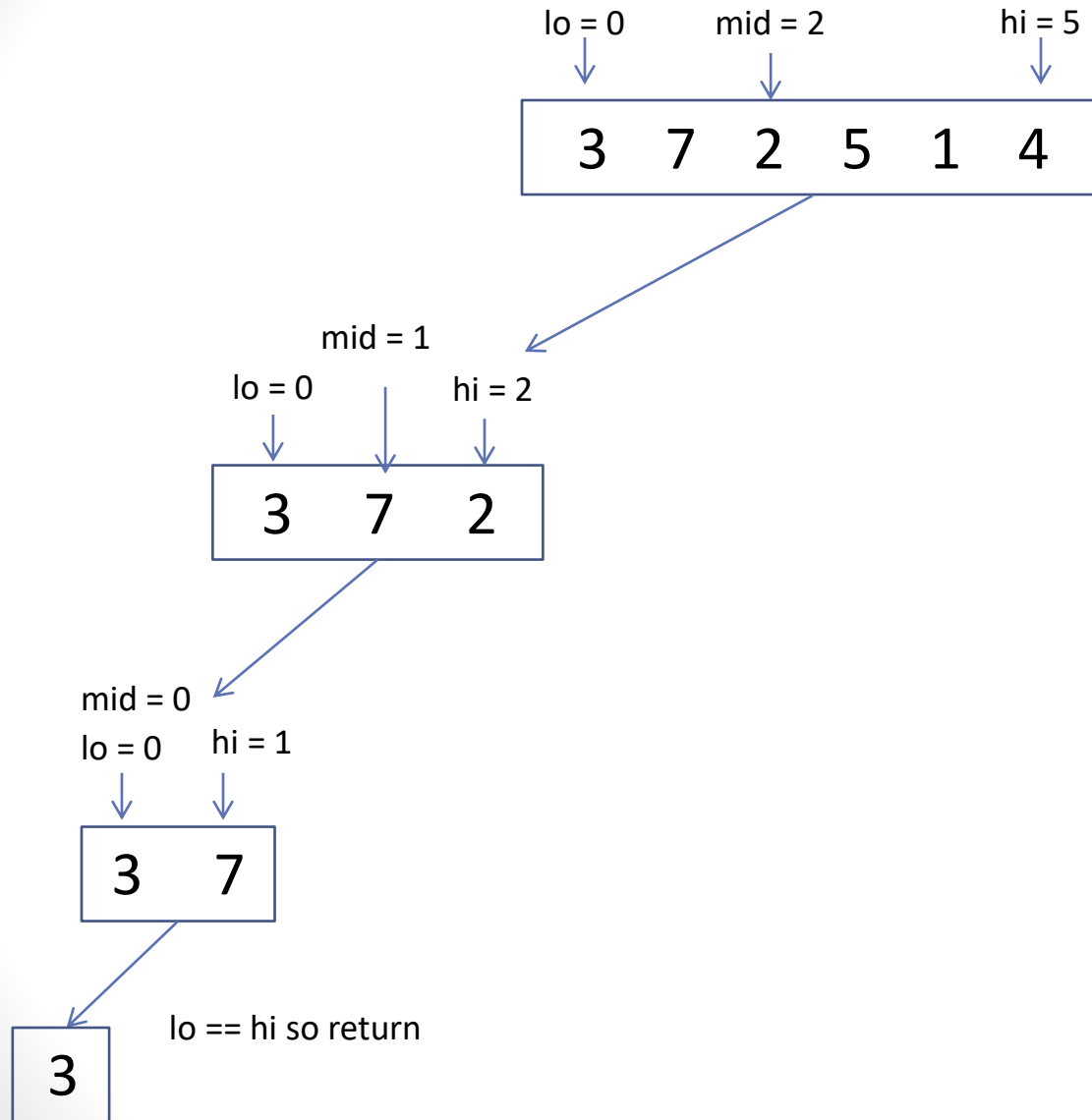
Merge Sort : An example



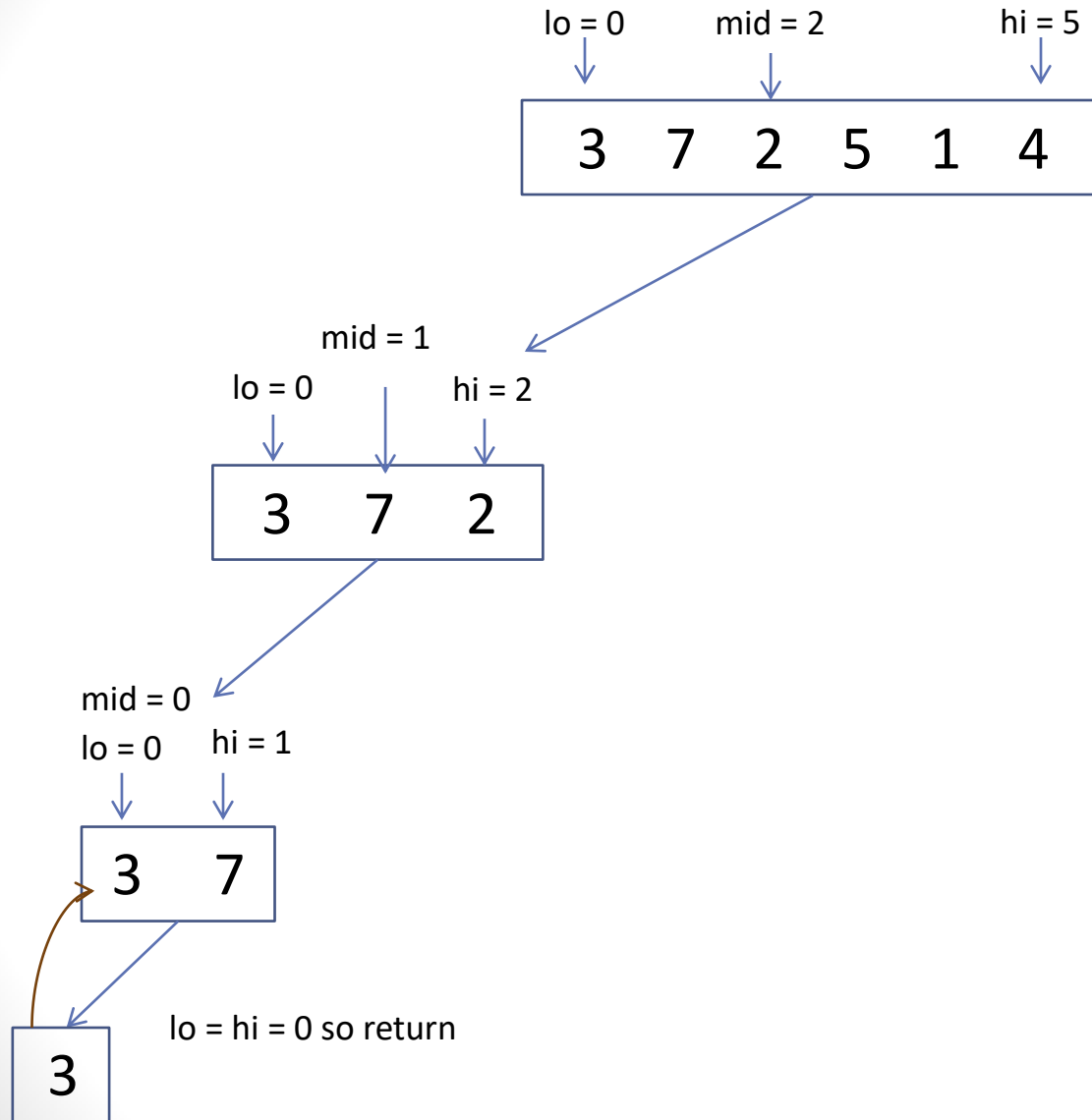
Merge Sort : An example



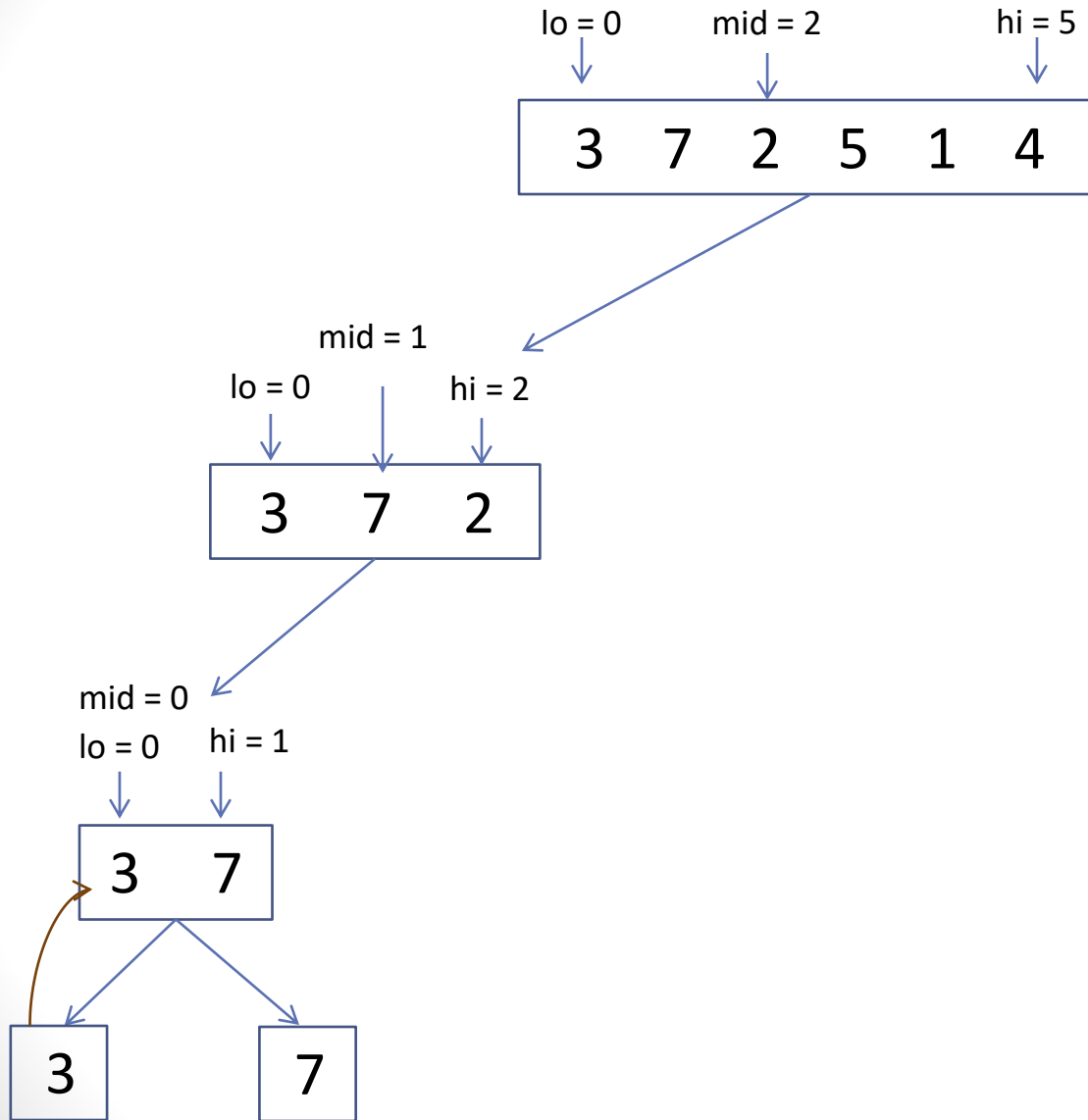
Merge Sort : An example



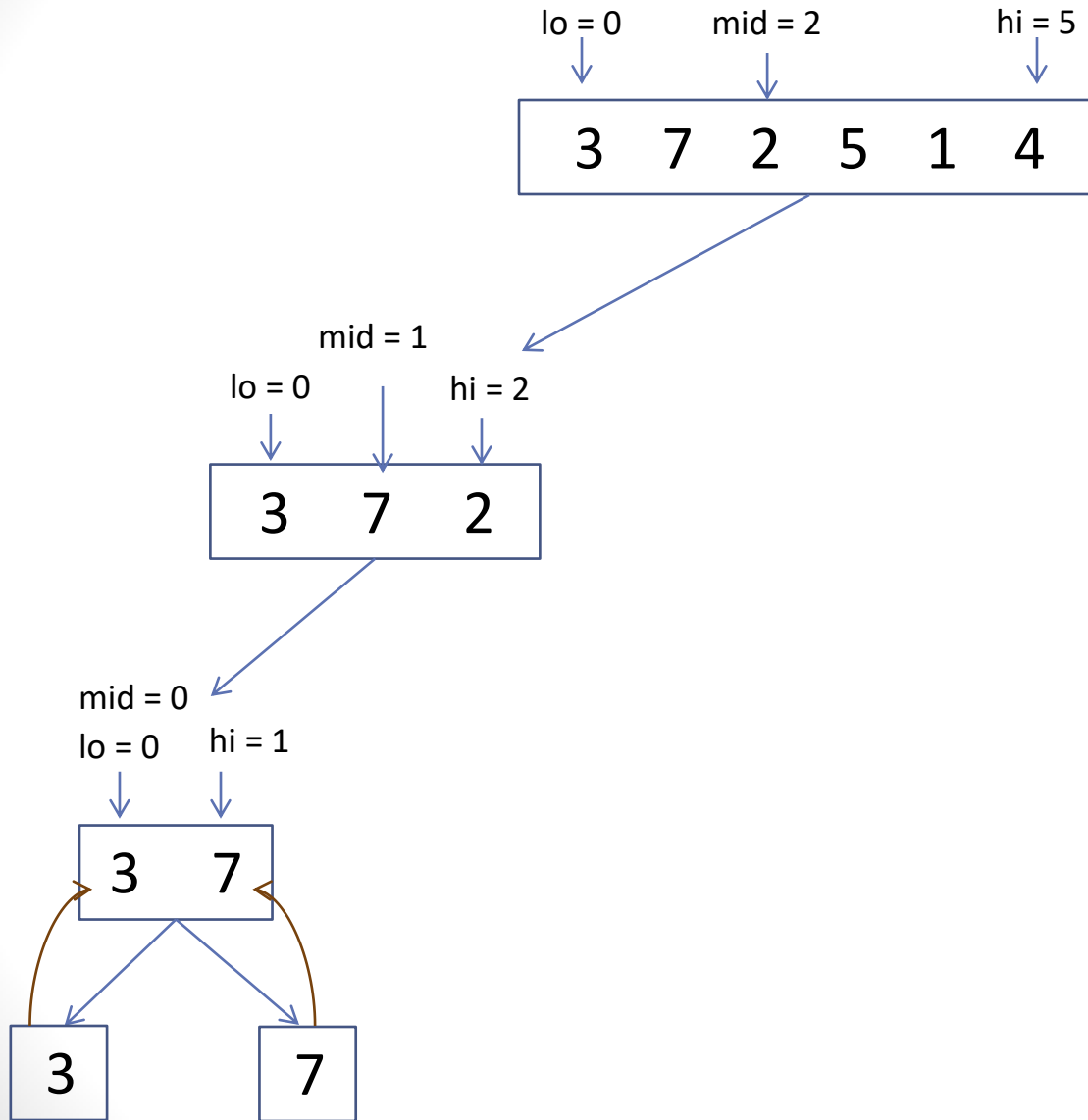
Merge Sort : An example



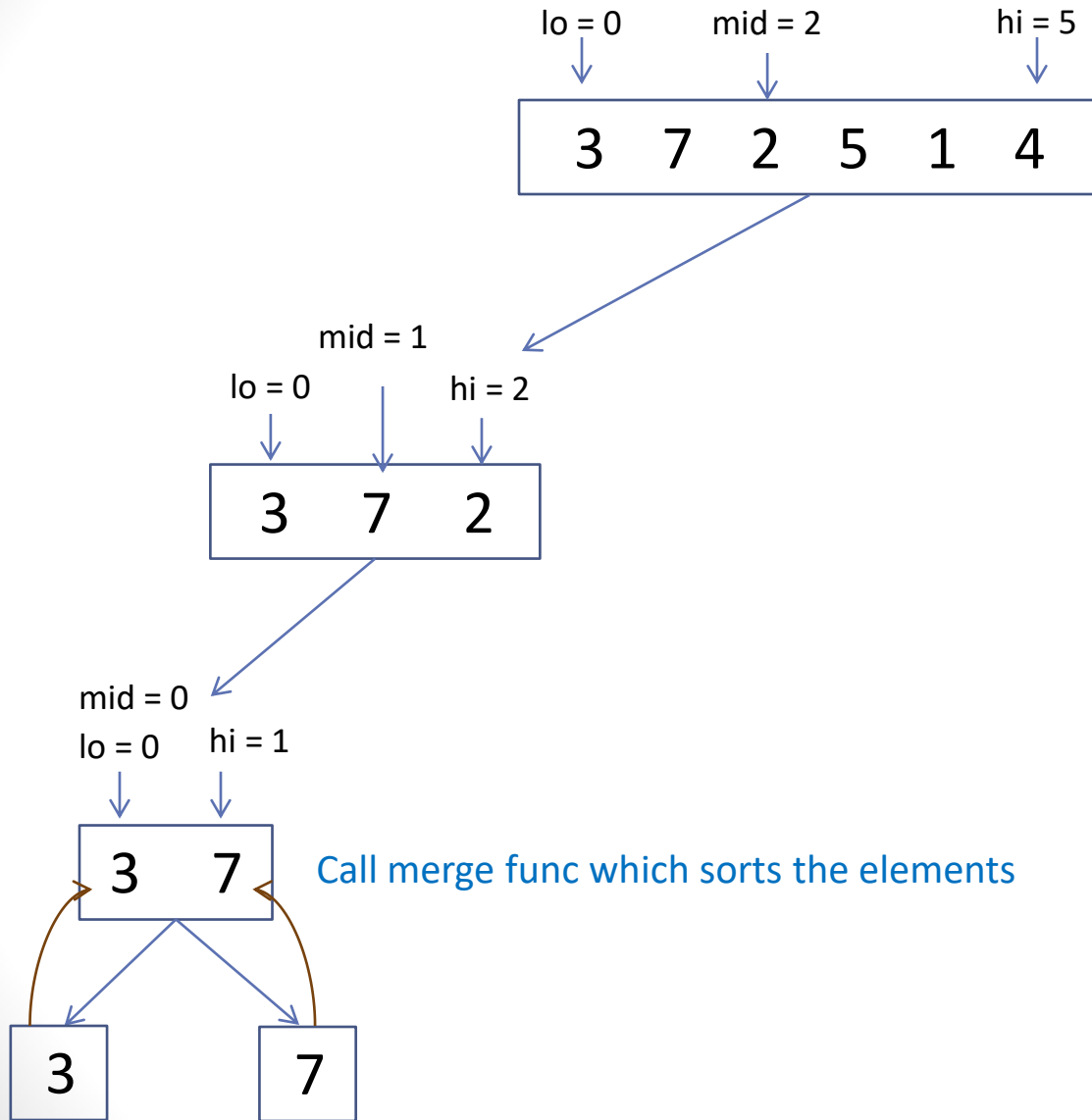
Merge Sort : An example



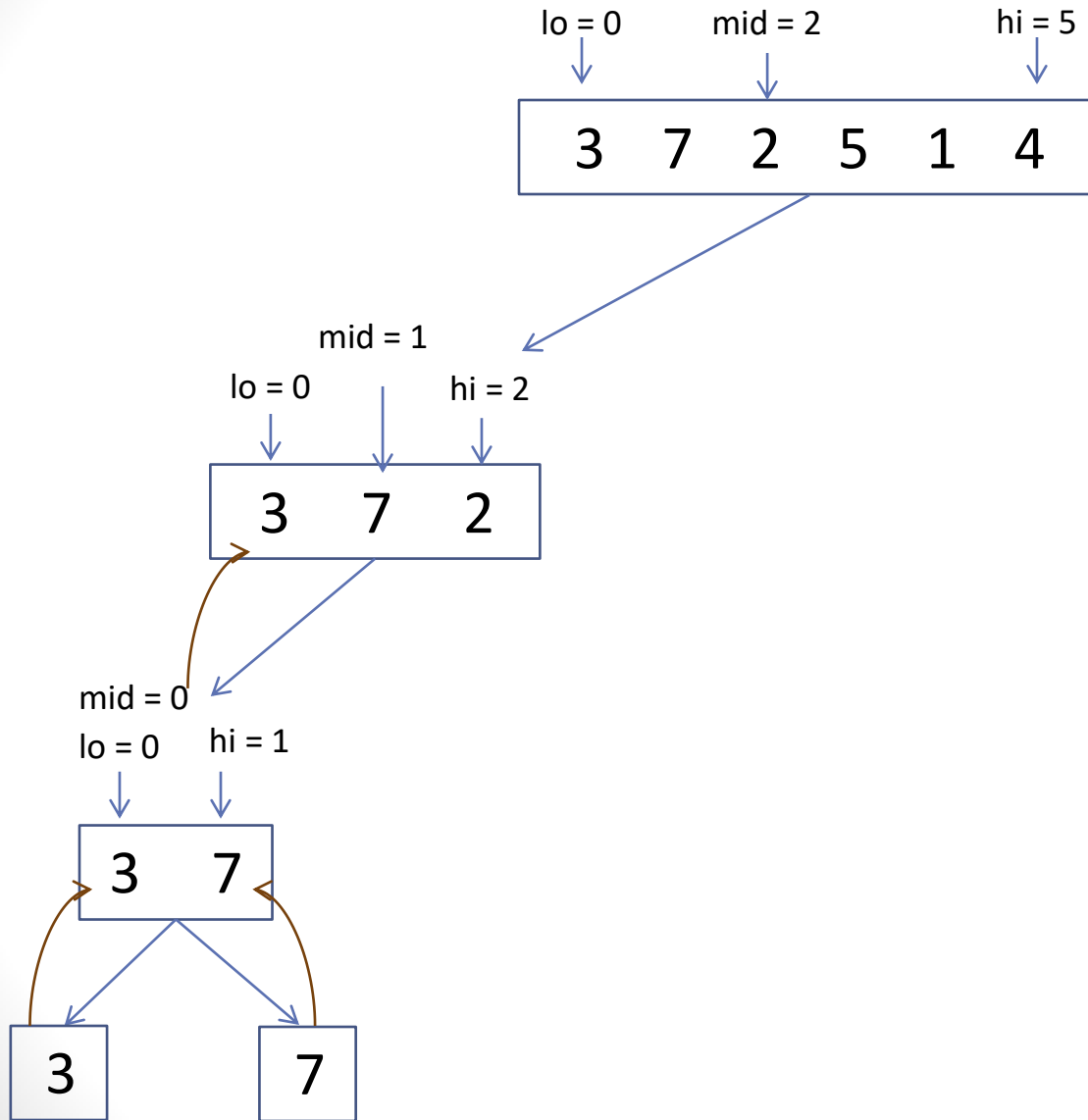
Merge Sort : An example



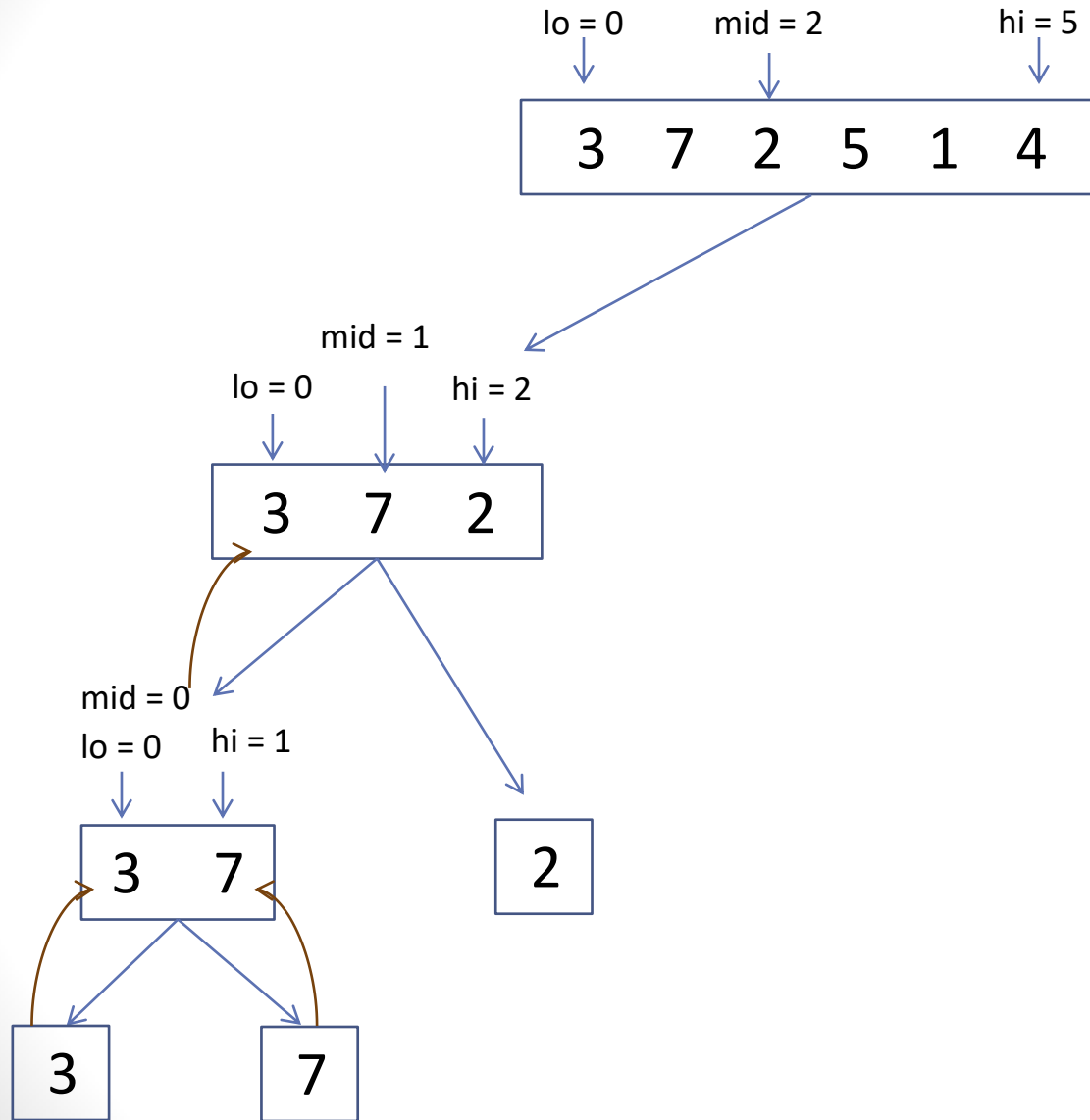
Merge Sort : An example



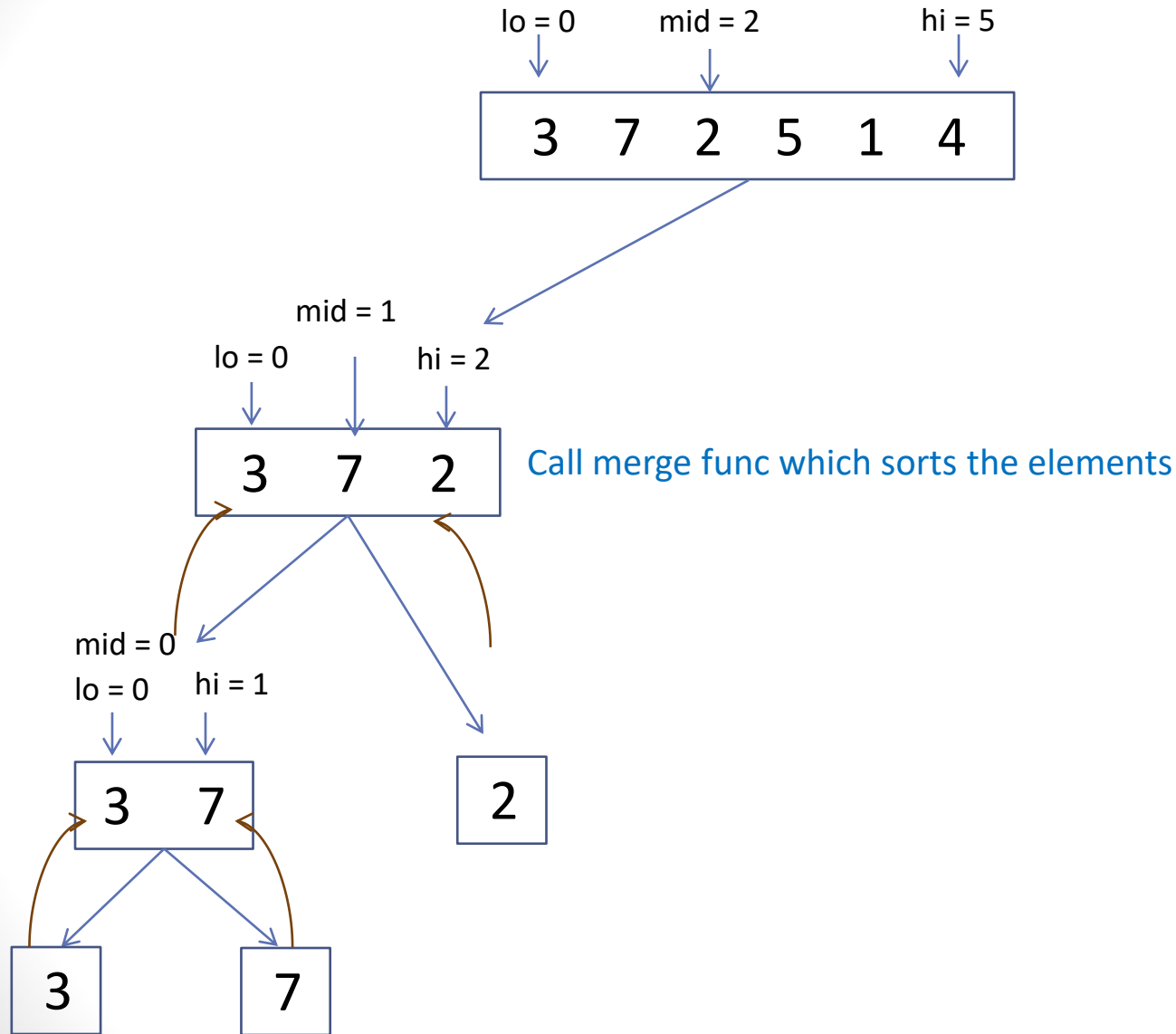
Merge Sort : An example



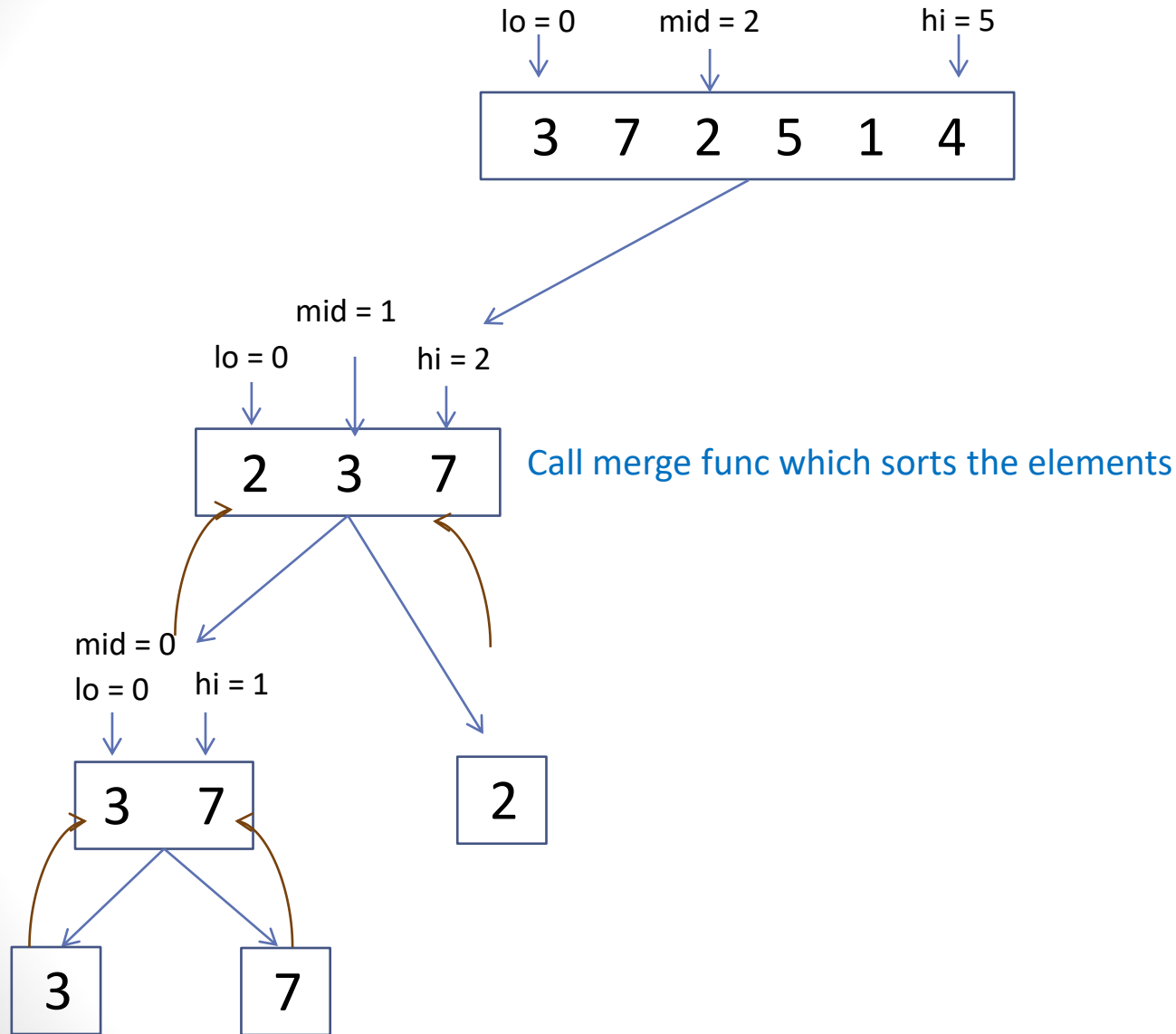
Merge Sort : An example



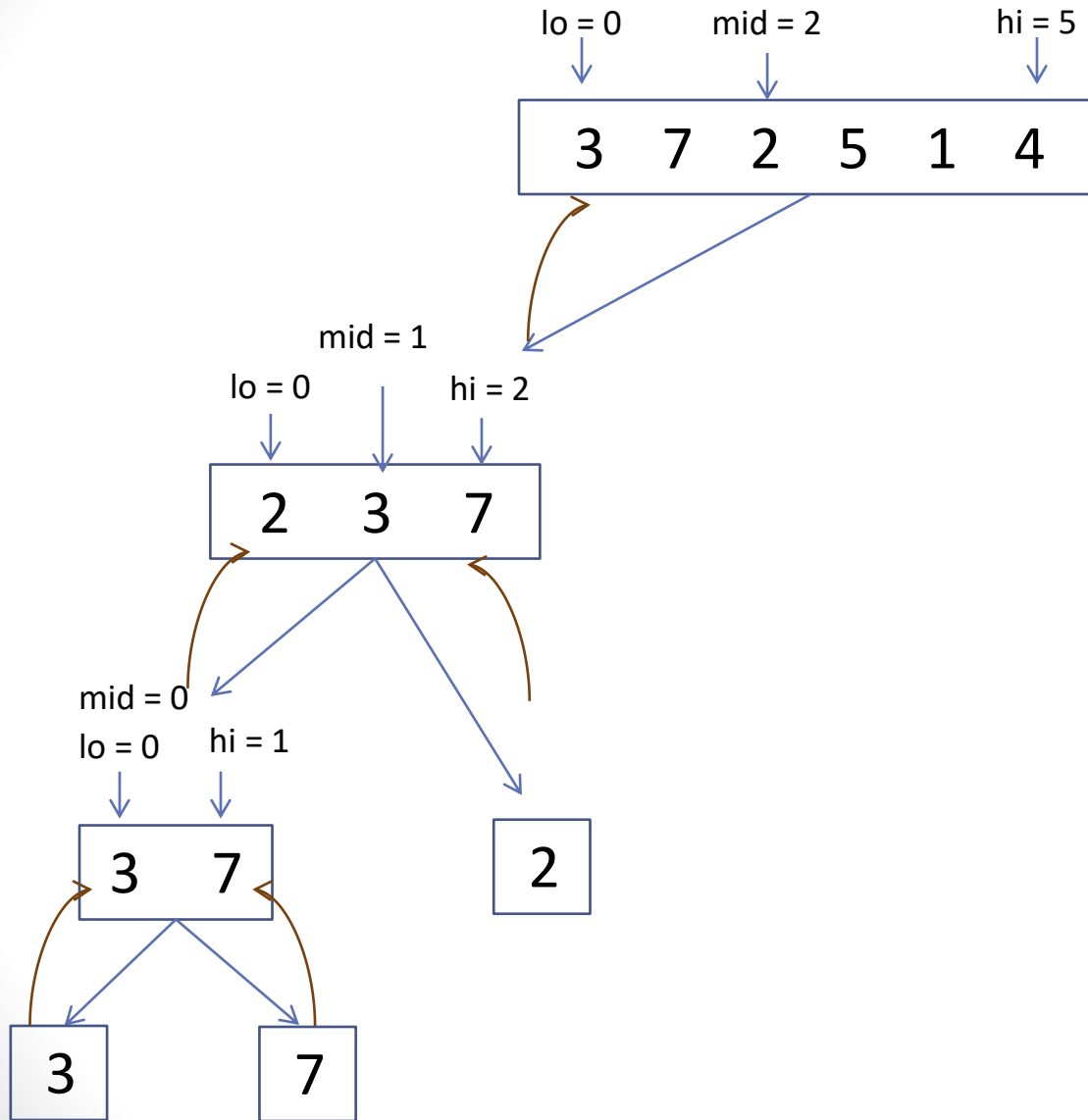
Merge Sort : An example



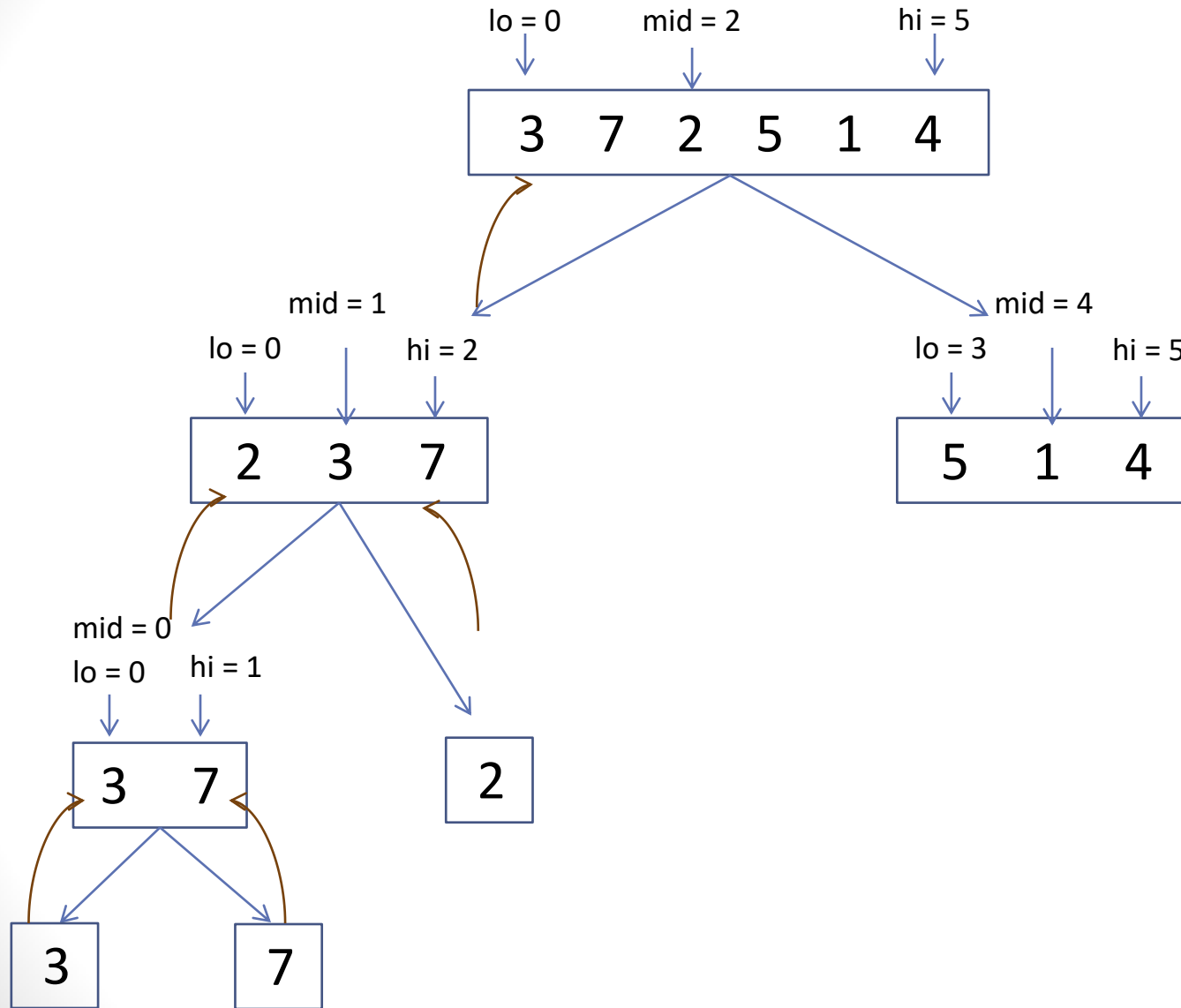
Merge Sort : An example



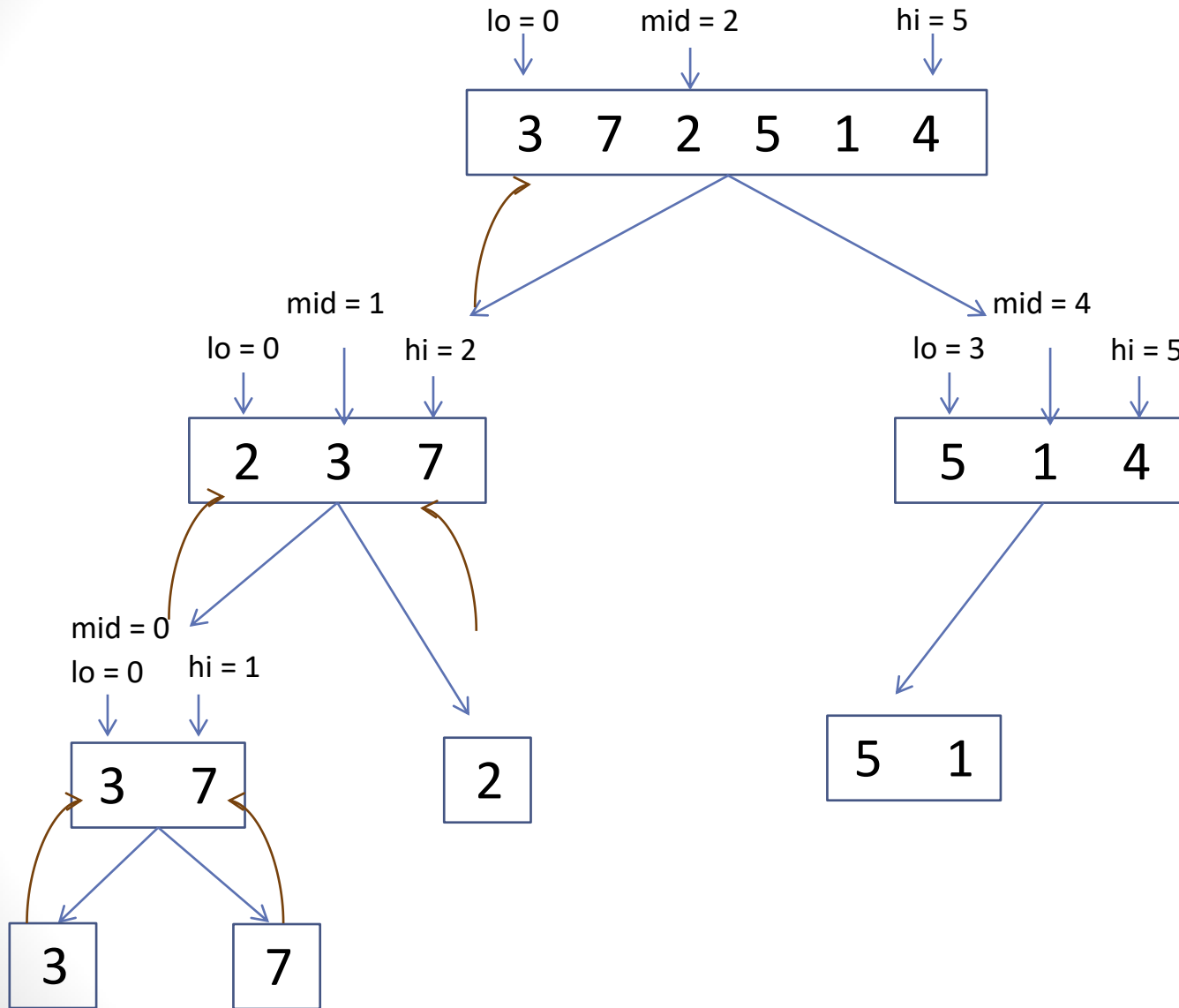
Merge Sort : An example



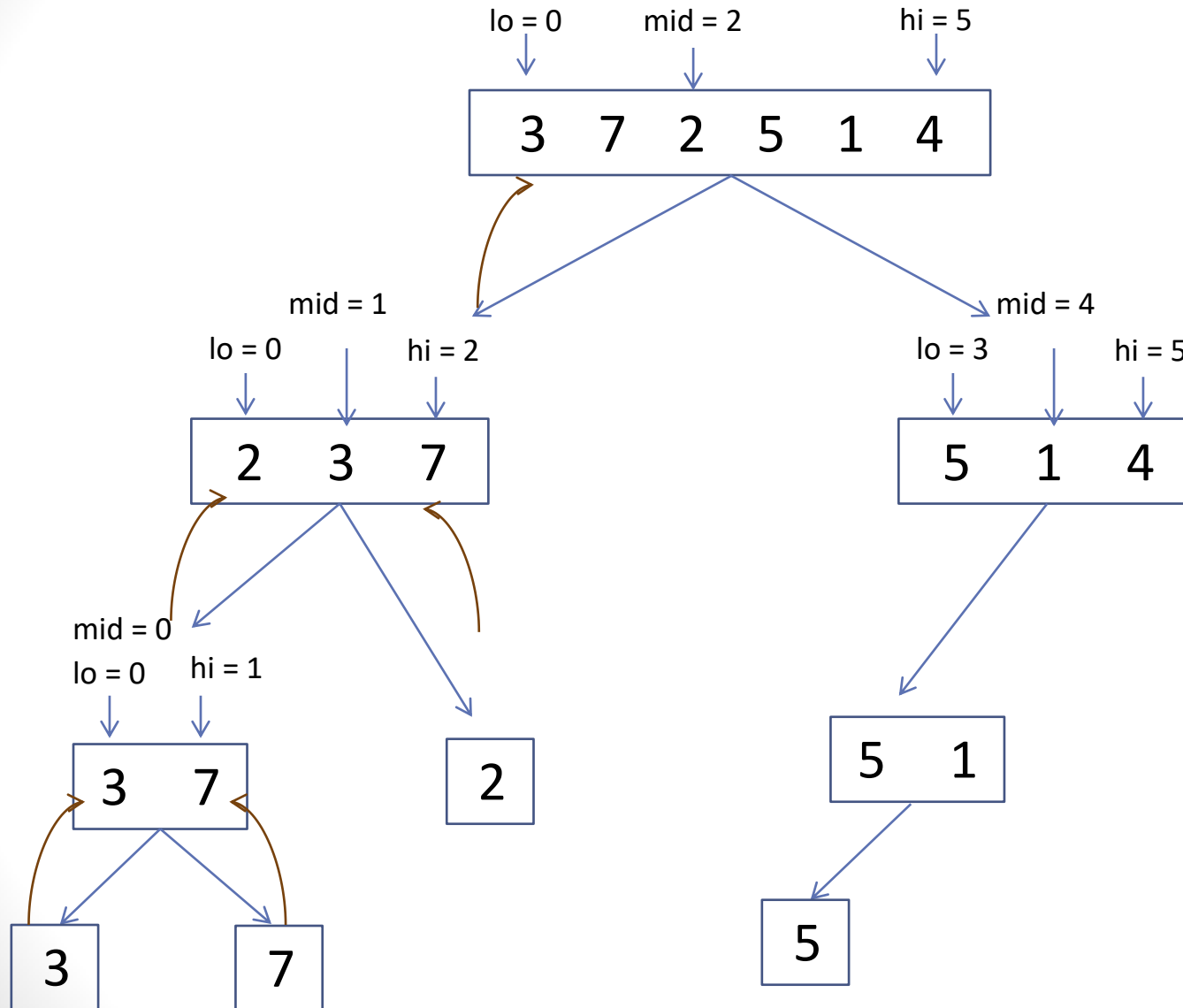
Merge Sort : An example



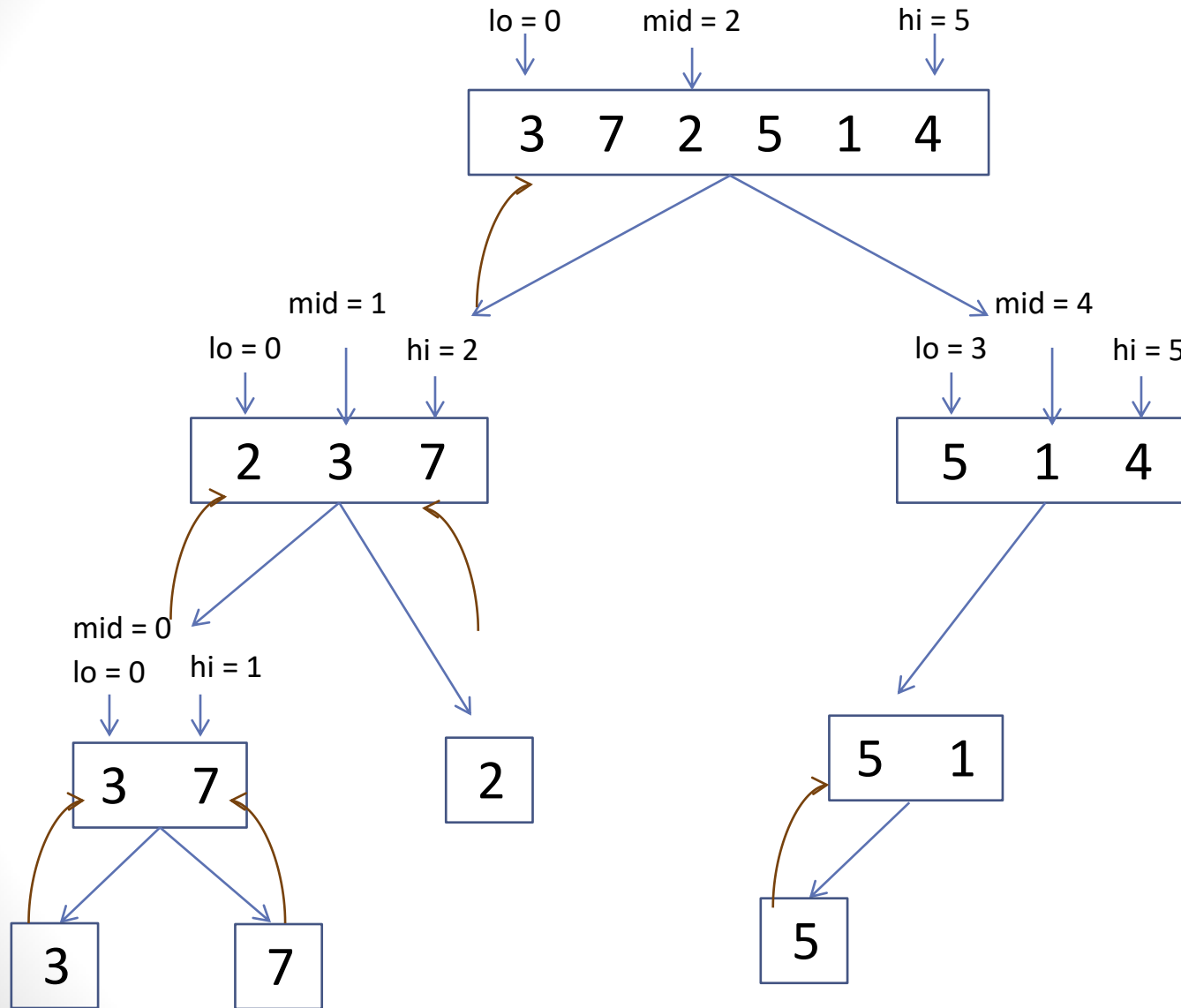
Merge Sort : An example



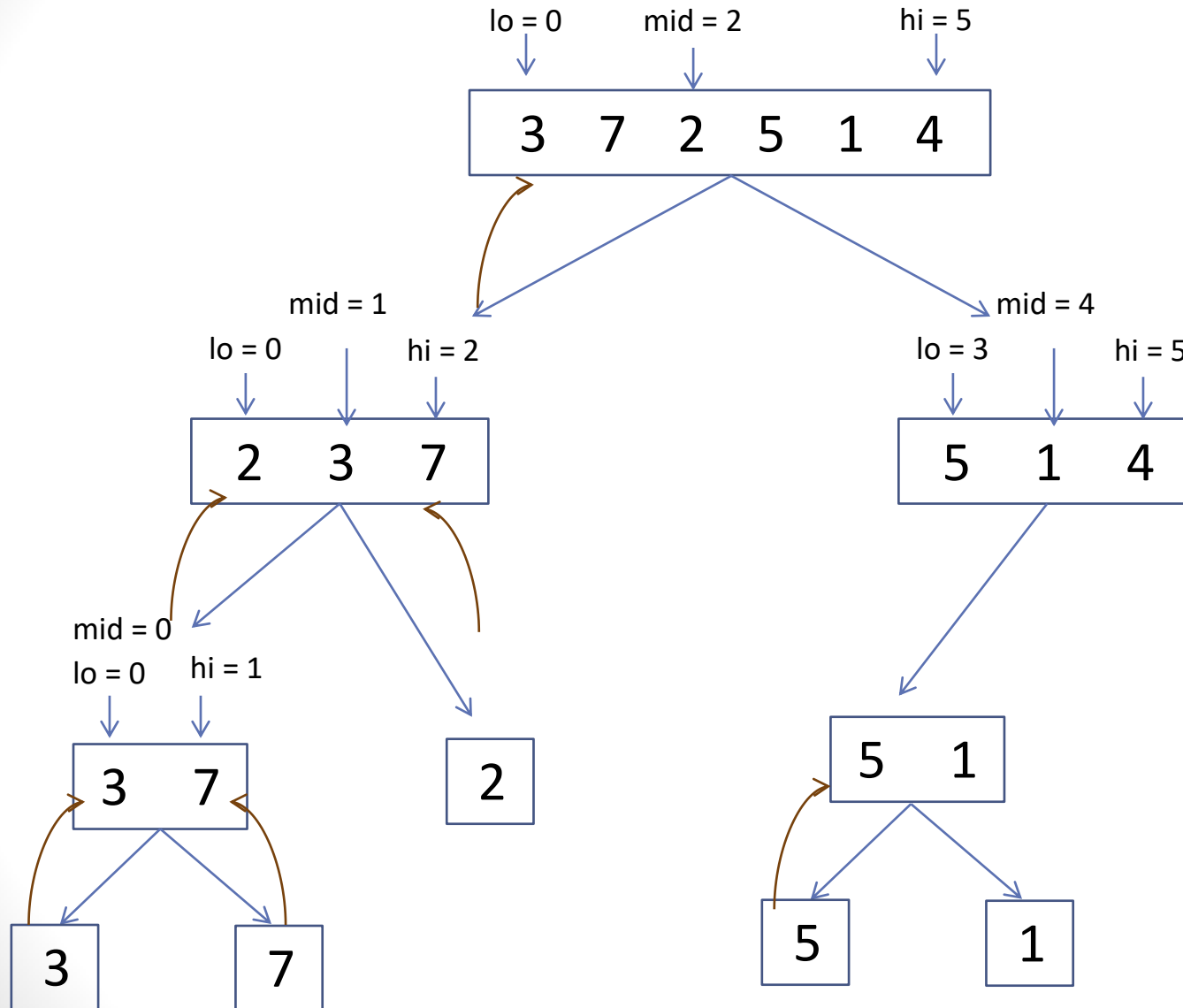
Merge Sort : An example



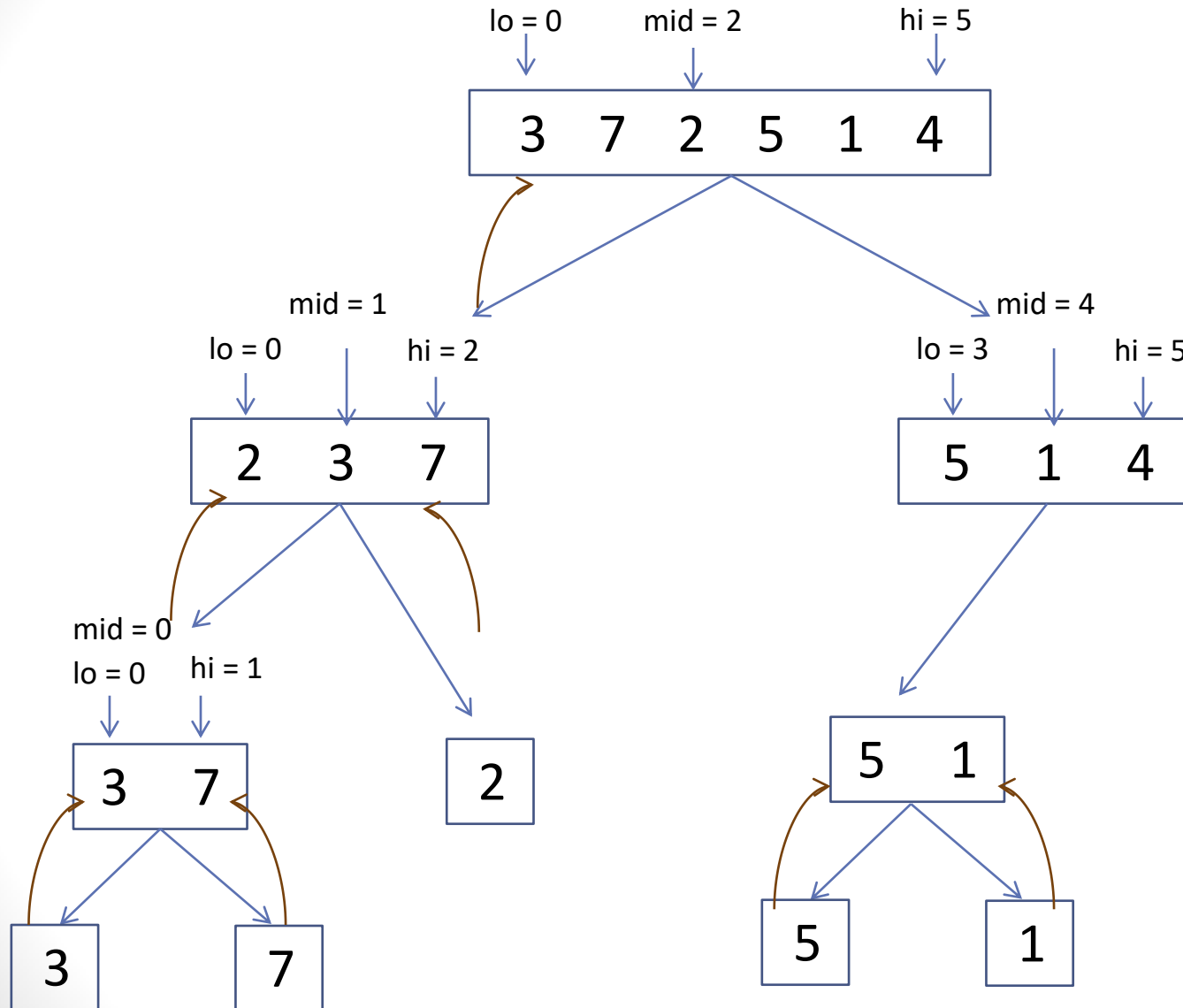
Merge Sort : An example



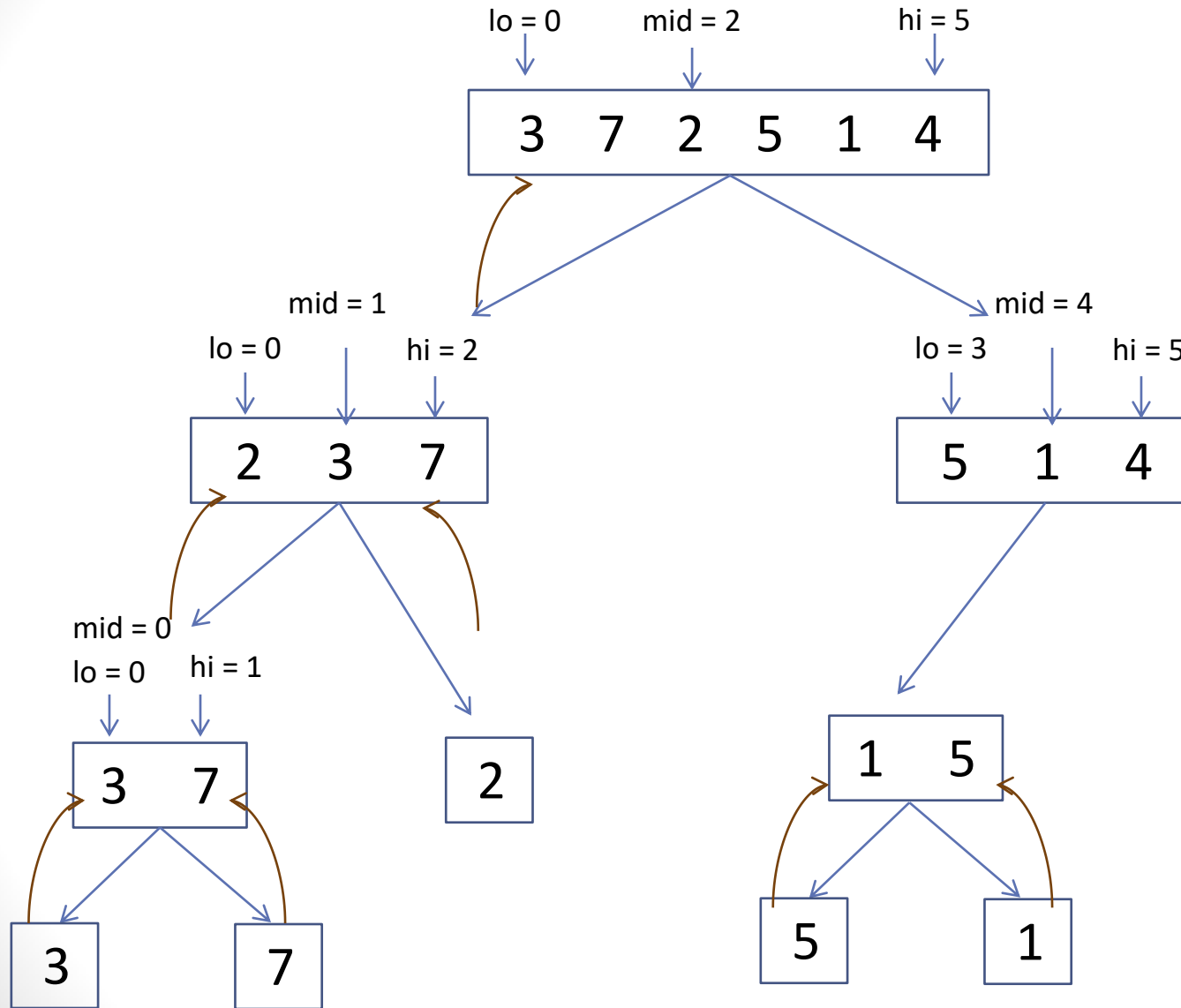
Merge Sort : An example



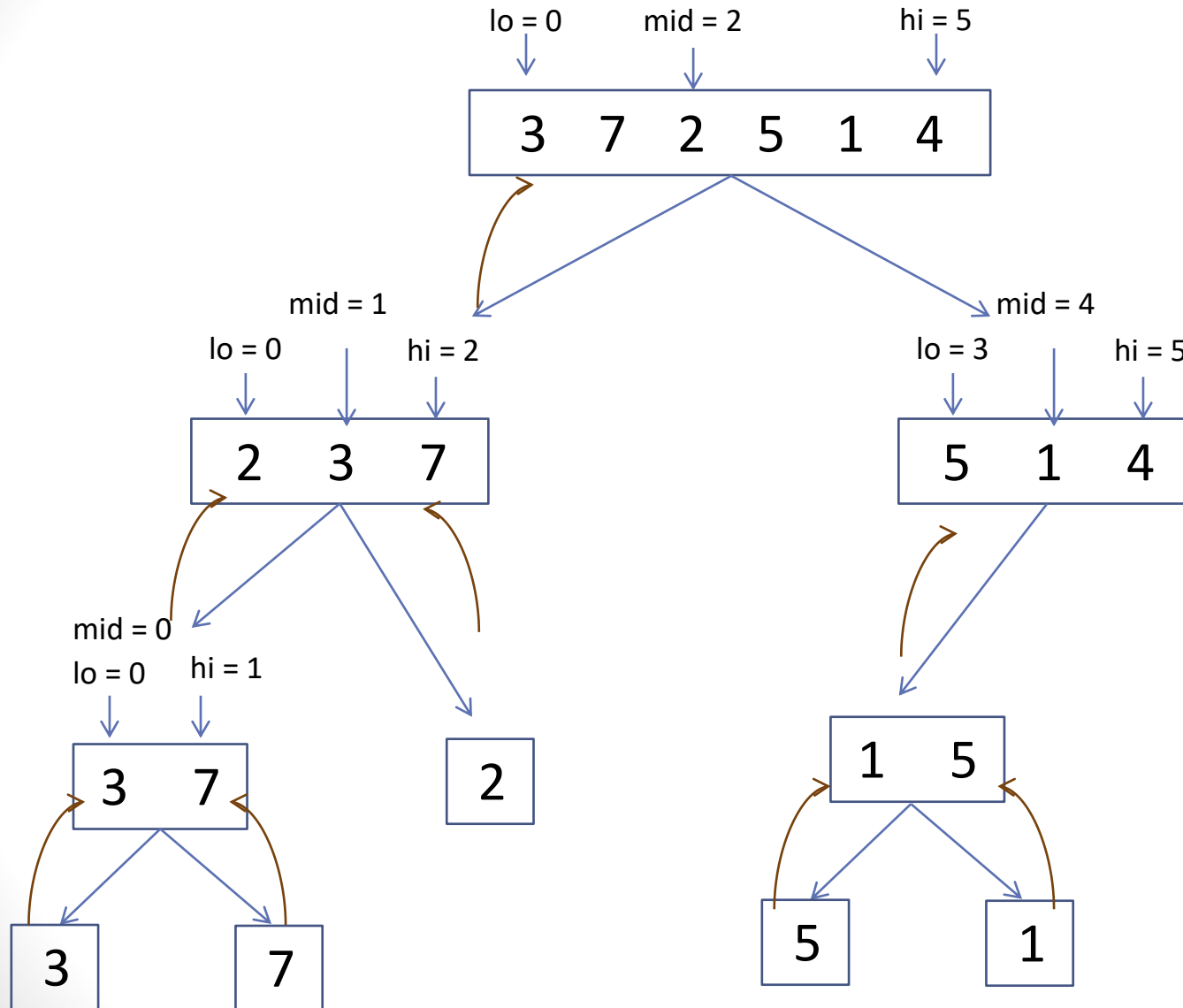
Merge Sort : An example



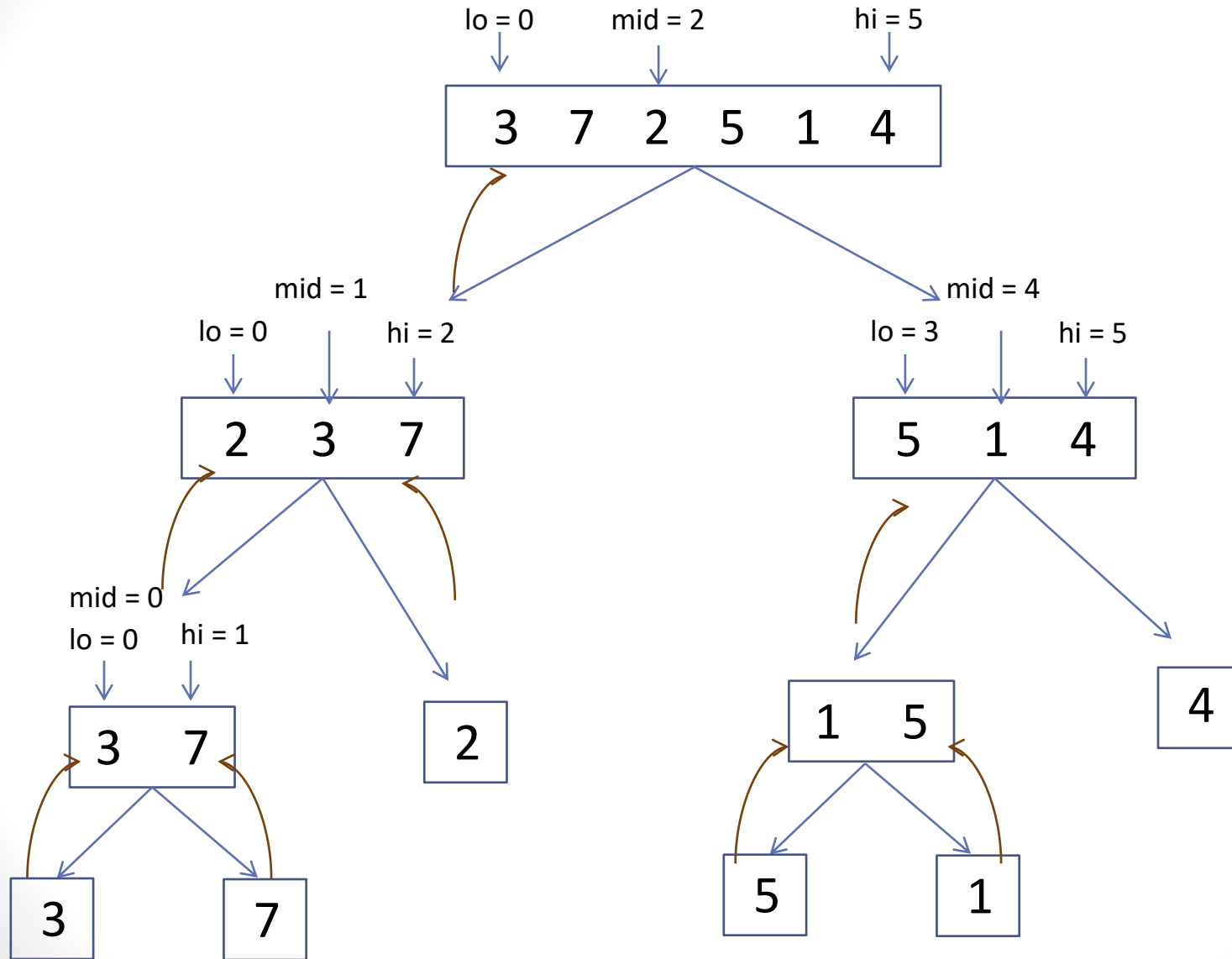
Merge Sort : An example



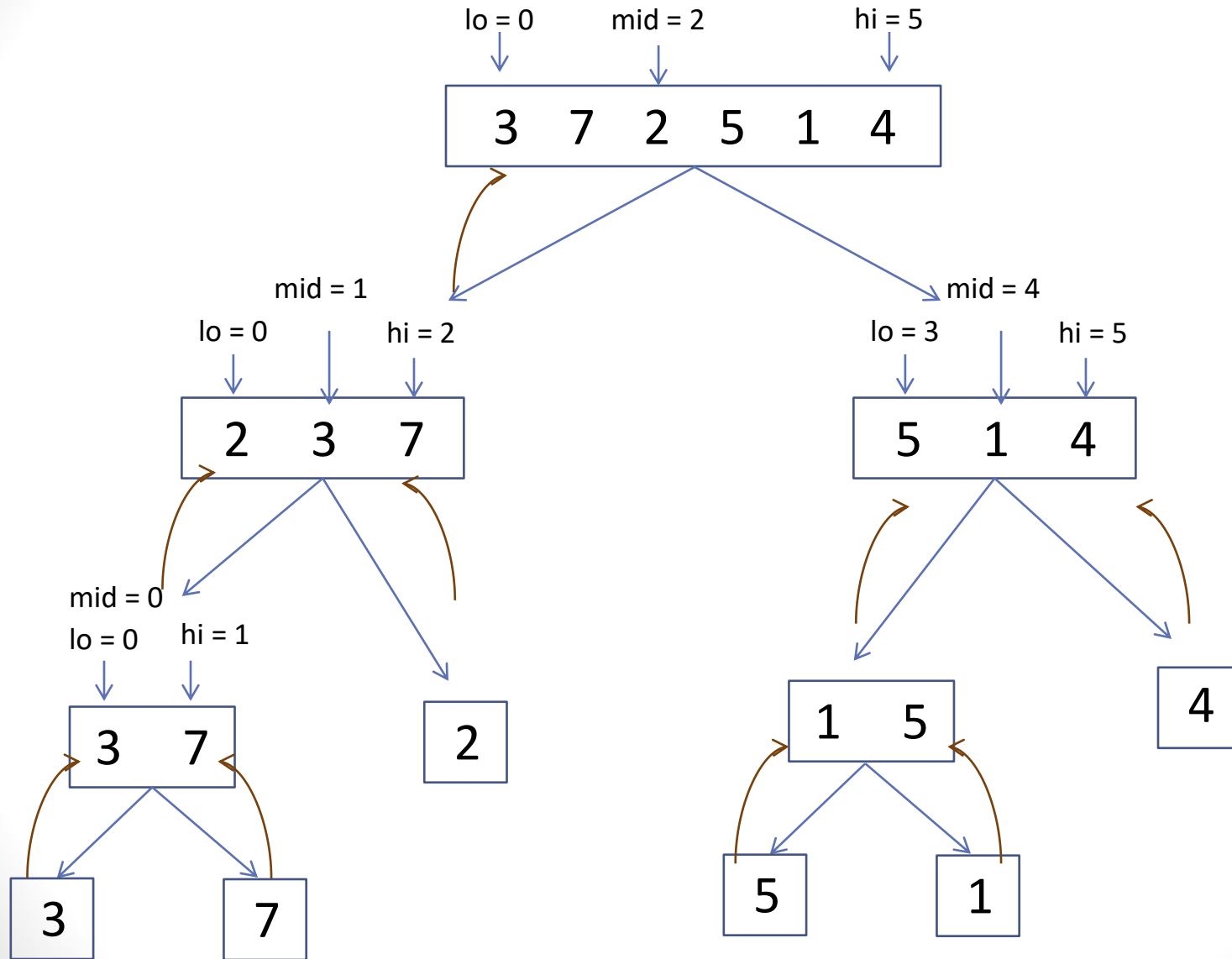
Merge Sort : An example



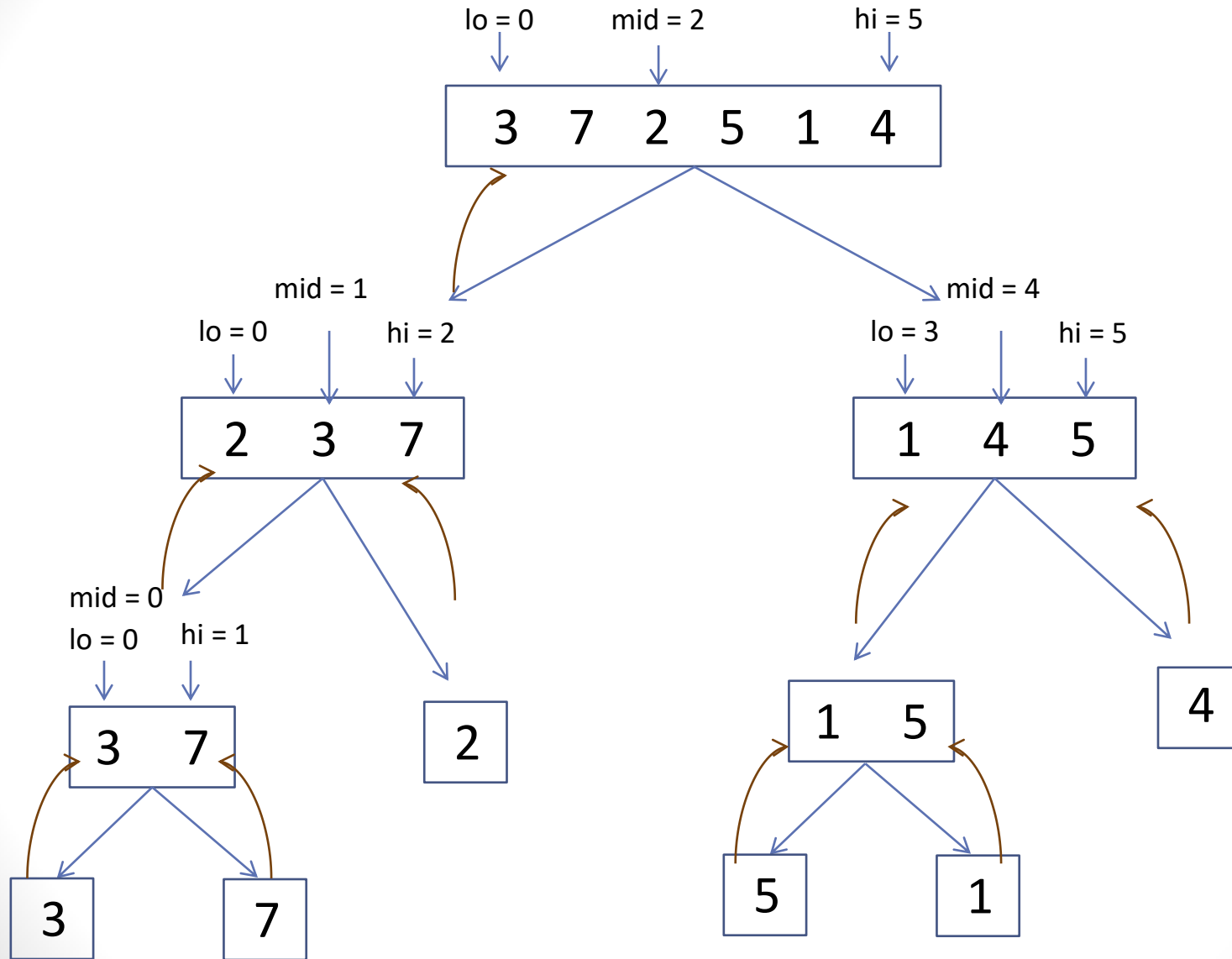
Merge Sort : An example



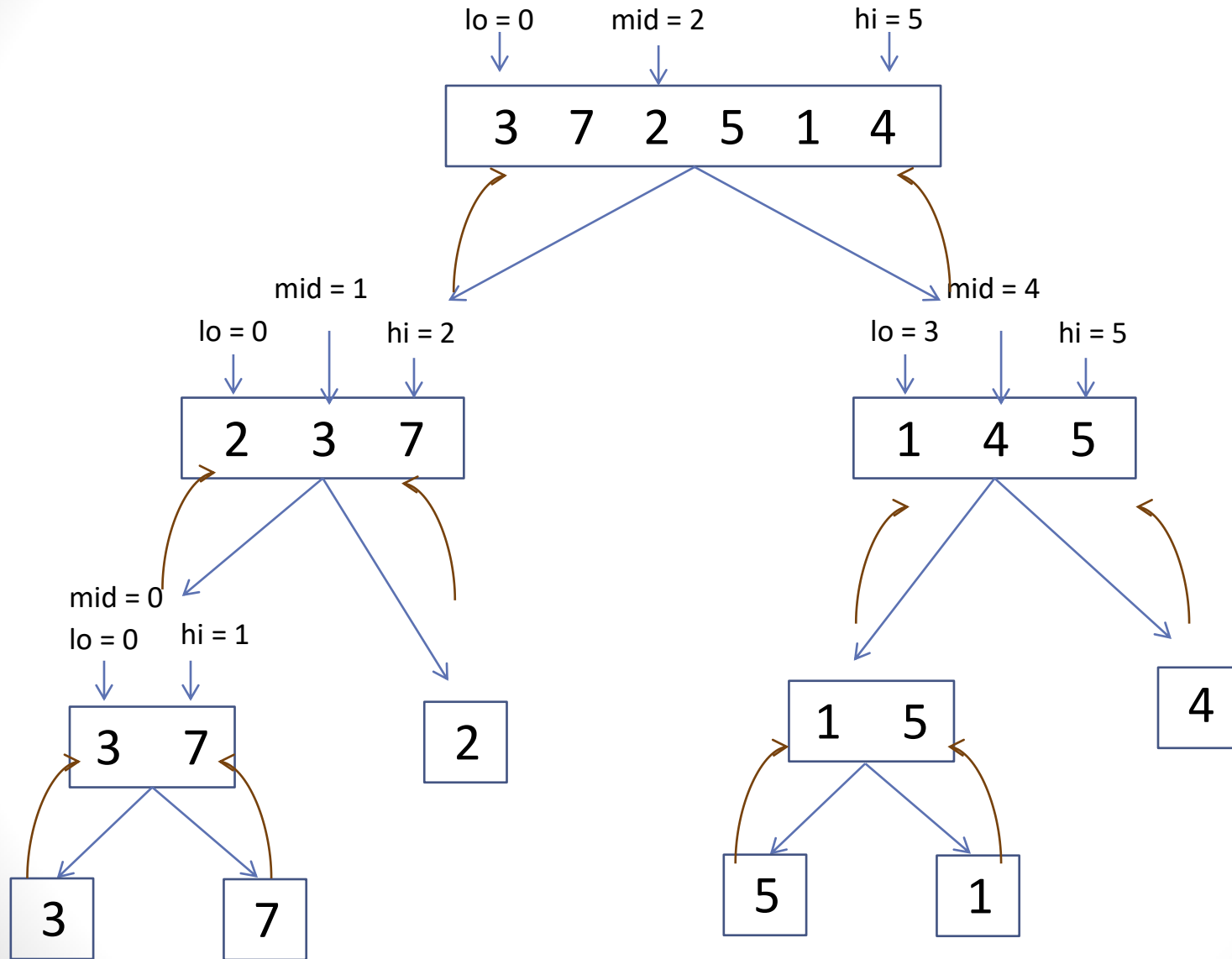
Merge Sort : An example



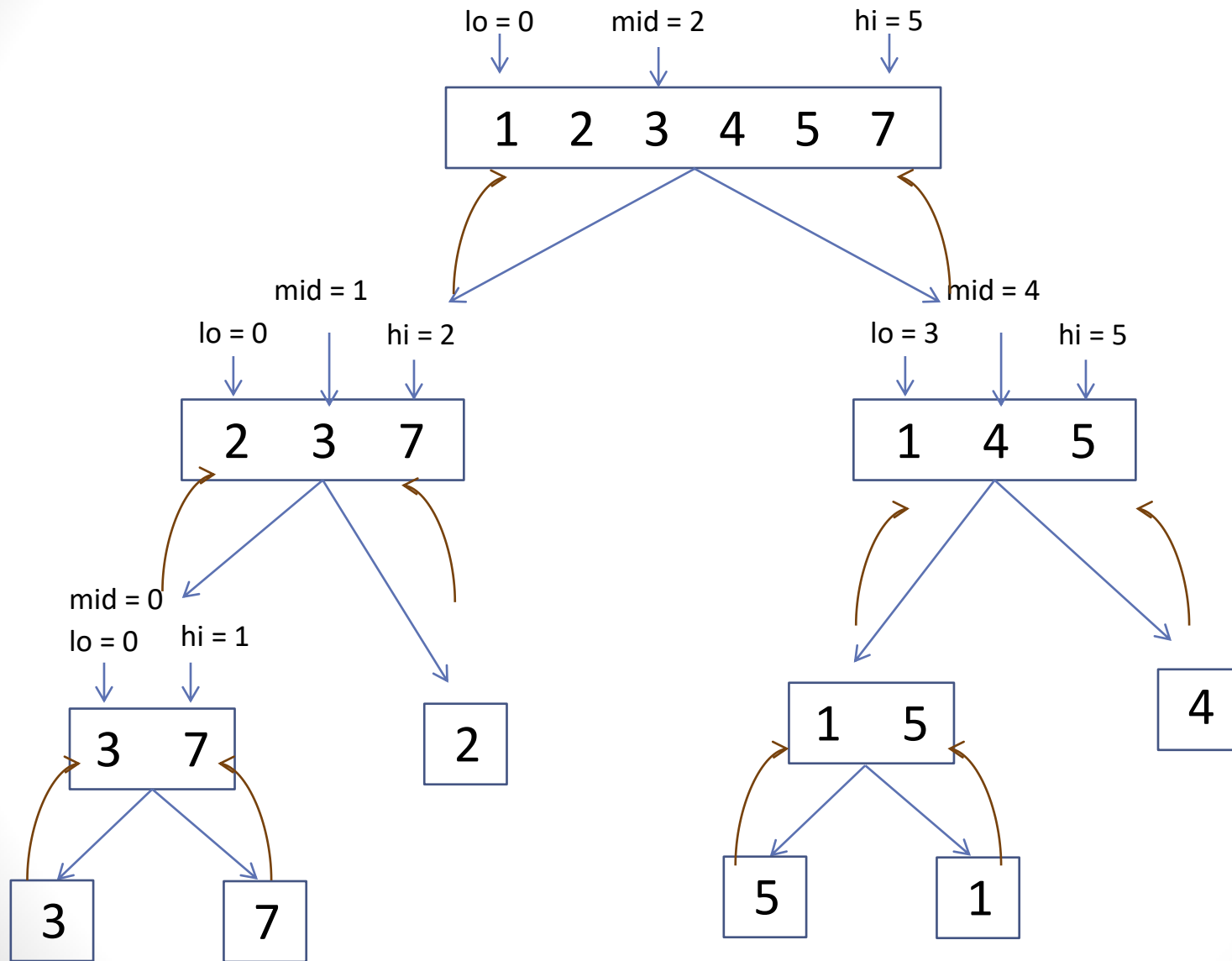
Merge Sort : An example



Merge Sort : An example



Merge Sort : An example



Merge Sort : Code

```
void mergeSort(int arr[], int lo, int hi) {  
    if (lo < hi) {  
        int mid = lo + (hi - lo) / 2;  
  
        mergeSort(arr, lo, mid);  
        mergeSort(arr, mid + 1, hi);  
  
        merge(arr, lo, mid, hi);  
    }  
}
```

Merge Sort: Merge function

```
void merge(int arr[], int lo, int mid, int hi) {
    int n1 = mid - lo + 1;
    int n2 = hi - mid;

    vector<int> left(n1), right(n2);

    // copy left half
    for (int i = 0; i < n1; i++) {
        left[i] = arr[lo + i];
    }

    // copy right half
    for (int j = 0; j < n2; j++) {
        right[j] = arr[mid + 1 + j];
    }

    int i = 0, j = 0, k = lo;

    // merge the two halves
    while (i < n1 && j < n2) {
        if (left[i] <= right[j]) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
        }
    }

    // copy remaining left elements
    while (i < n1) {
        arr[k++] = left[i++];
    }

    // copy remaining right elements
    while (j < n2) {
        arr[k++] = right[j++];
    }
}
```


Merge Sort :Merge function

```
void merge(int arr[], int lo, int mid, int hi) {  
    int n1 = mid - lo + 1;  
    int n2 = hi - mid;  
  
    vector<int> left(n1), right(n2);  
  
    // copy left half  
    for (int i = 0; i < n1; i++) {  
        left[i] = arr[lo + i];  
    }  
  
    // copy right half  
    for (int j = 0; j < n2; j++) {  
        right[j] = arr[mid + 1 + j];  
    }  
}
```

Merge Sort : Merge function

```
int i = 0, j = 0, k = lo;

// merge the two halves
while (i < n1 && j < n2) {
    if (left[i] <= right[j]) {
        arr[k++] = left[i++];
    } else {
        arr[k++] = right[j++];
    }
}
```

Merge Sort : Merge function

```
// copy remaining left elements
while (i < n1) {
    arr[k++] = left[i++];
}

// copy remaining right elements
while (j < n2) {
    arr[k++] = right[j++];
}
}
```

Merge Sort : Merge Operation



merged arr

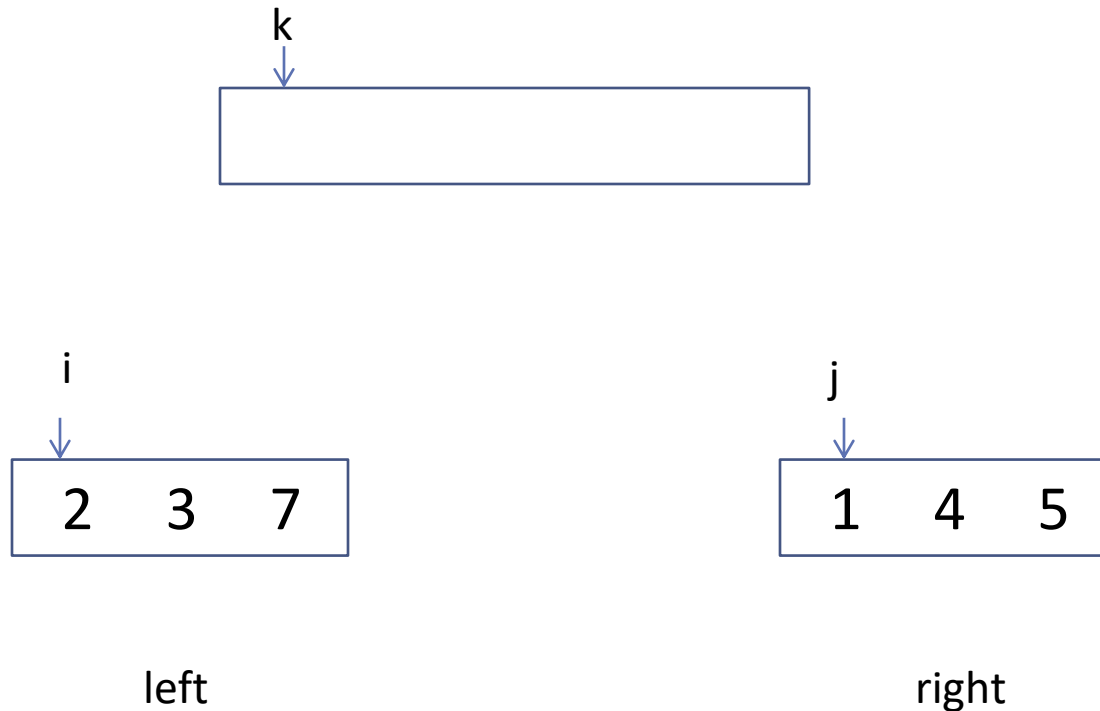


left

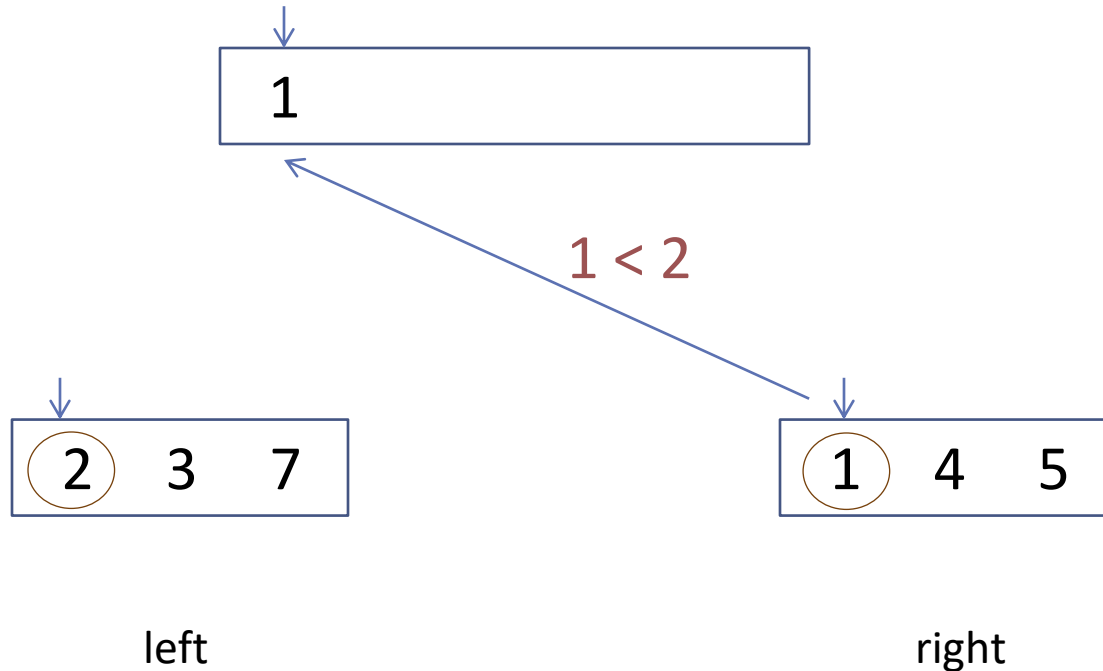


right

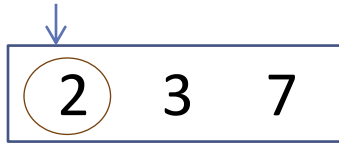
Merge Sort : Merge Operation



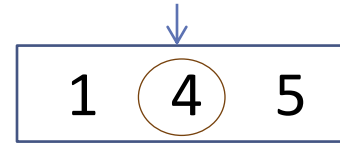
Merge Sort : Merge Operation



Merge Sort : Merge Operation

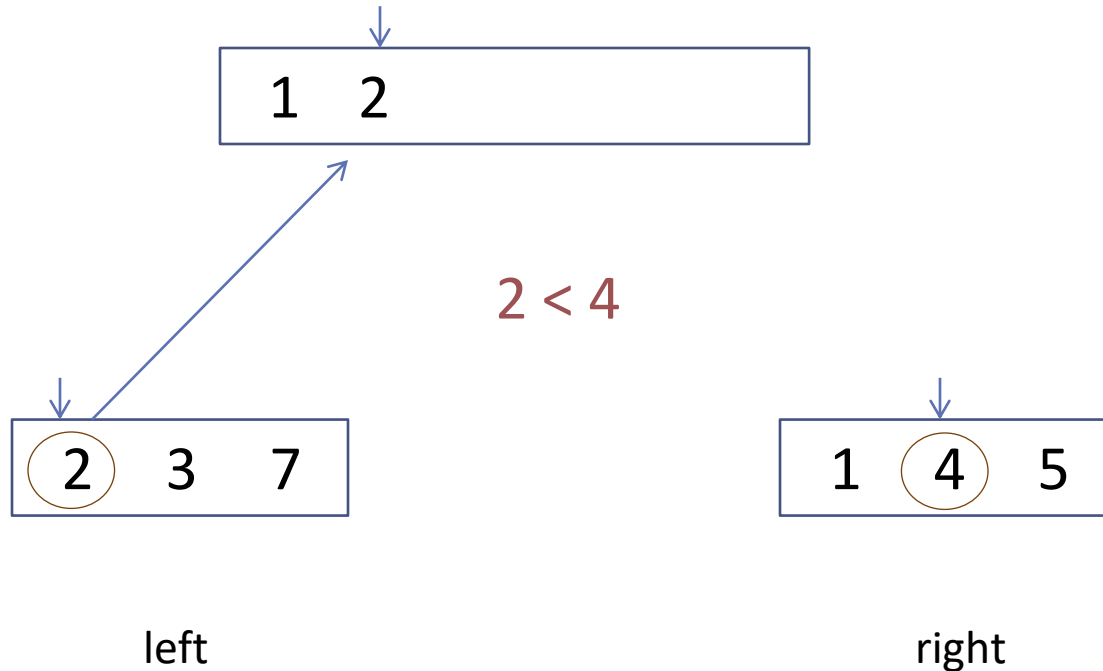


left

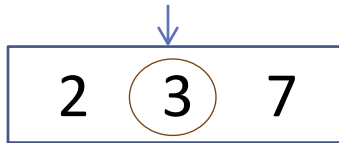


right

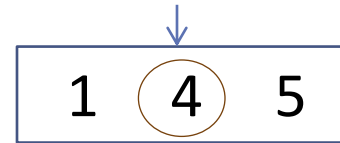
Merge Sort : Merge Operation



Merge Sort : Merge Operation

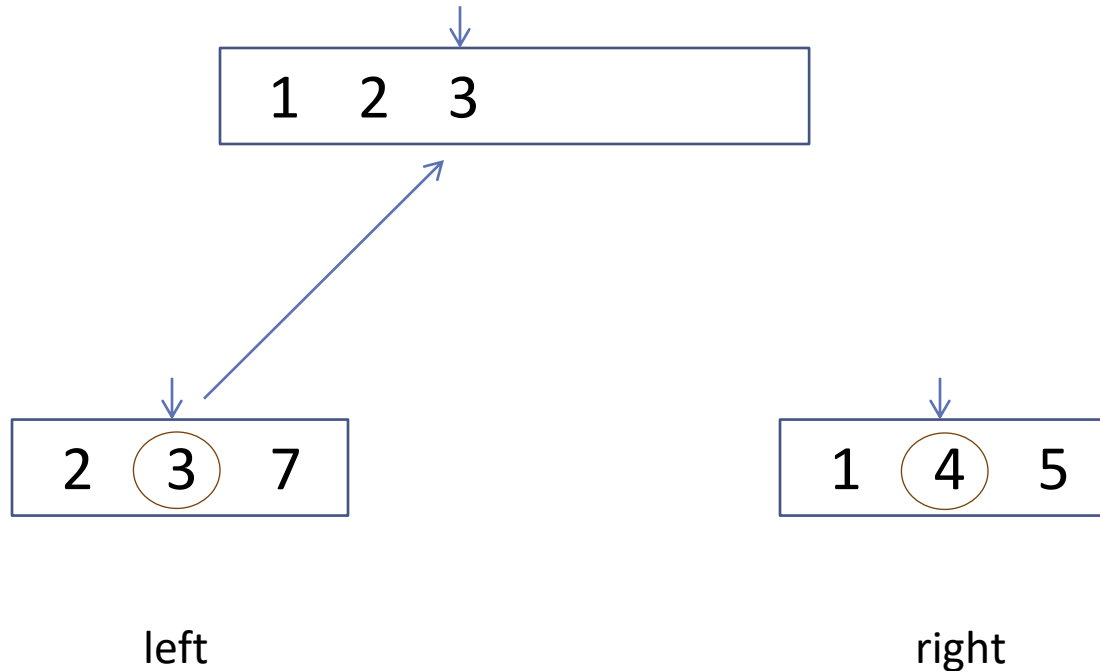


left

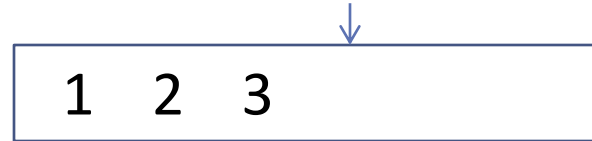


right

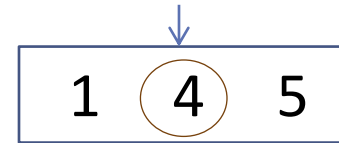
Merge Sort : Merge Operation



Merge Sort : Merge Operation

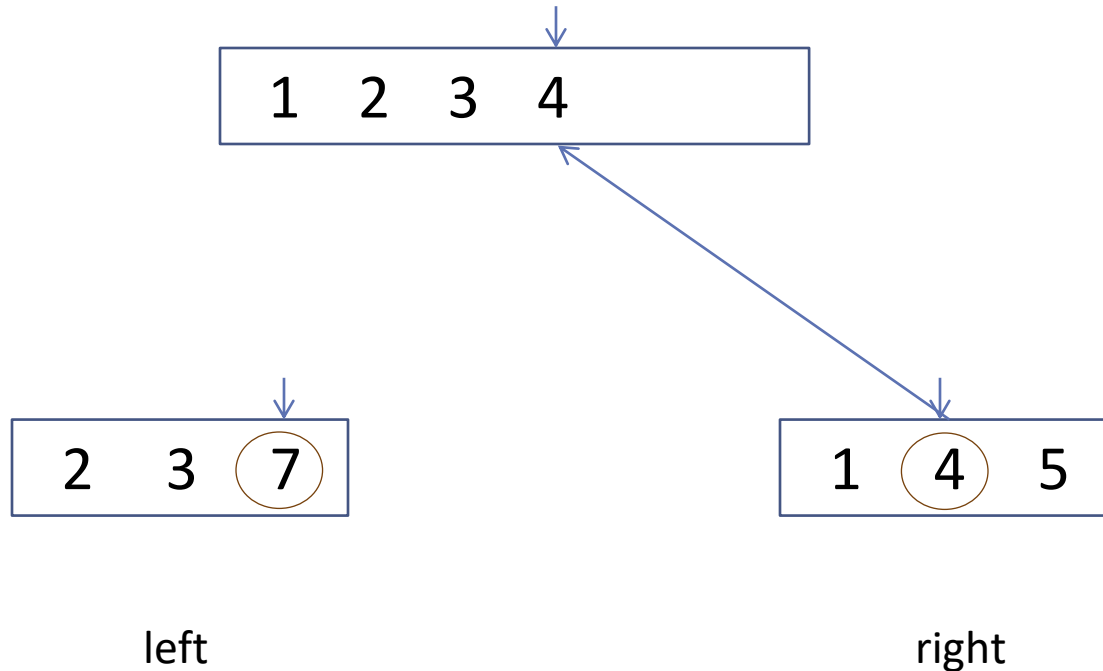


left

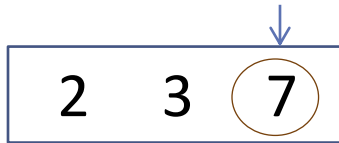


right

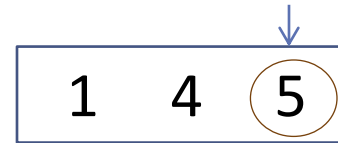
Merge Sort : Merge Operation



Merge Sort : Merge Operation

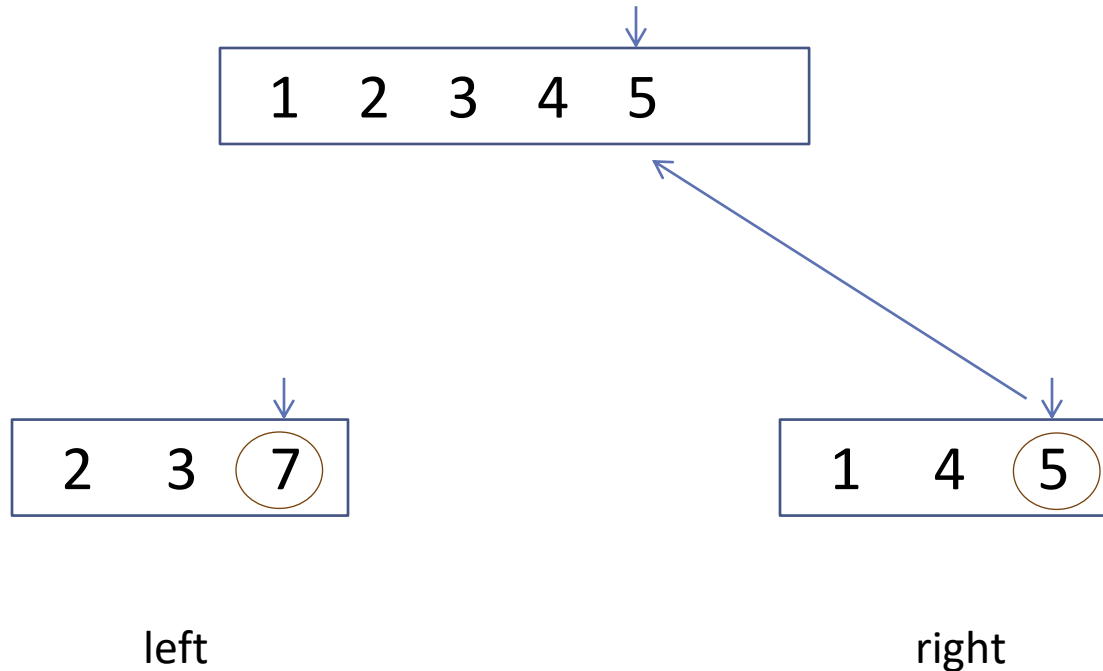


left



right

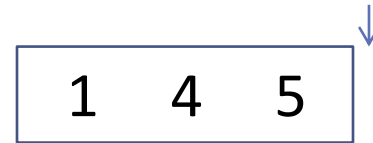
Merge Sort : Merge Operation



Merge Sort : Merge Operation

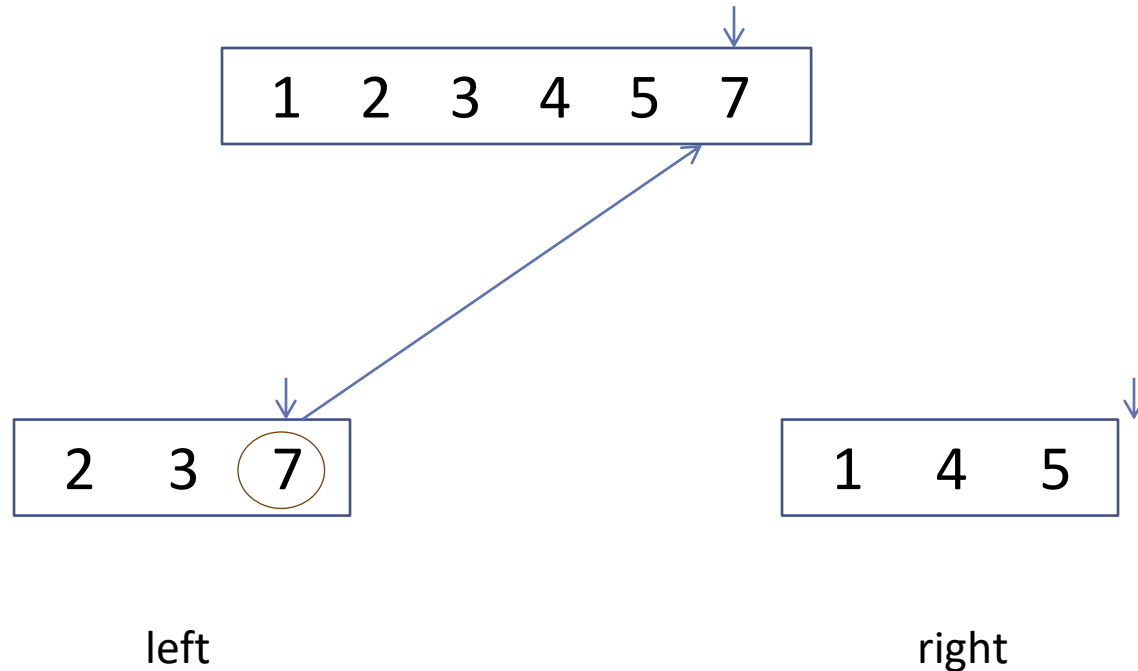


left

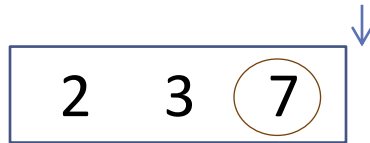


right

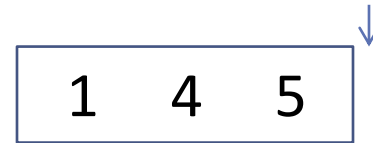
Merge Sort : Merge Operation



Merge Sort : Merge Operation



left



right

Merge Sort : Complexity

Time Complexity: $O(n \log n)$ for all cases

Auxiliary Space: $O(n)$